UML Diagrams: Complete Set

DormFix: Hostel Complaint and Maintenance Management System

This file contains all seven UML diagram types for the DormFix project: Use Case, Class, Object, Sequence, Collaboration, State and Activity. Every diagram is written in Mermaid syntax and was derived from the current source code under `server/` and `client/src/`.

To view a diagram, paste the Mermaid block into https://mermaid.live or open this file in a Mermaid-enabled editor (GitHub, GitLab, VS Code with the Mermaid extension, Obsidian). Export as PNG or SVG if the report needs static images.

Contents

1. Use Case Diagram
2. Class Diagram
3. Object Diagram
4. Sequence Diagrams
5. Collaboration Diagrams
6. State Diagrams
7. Activity Diagrams
8. Diagram-to-Source Mapping
9. Implementation Gaps Found During Modelling

---

1. Use Case Diagram

Two actors interact with the DormFix system. The Resident is a self-registered hostel student. The Warden is a seeded administrator account and cannot self-register.

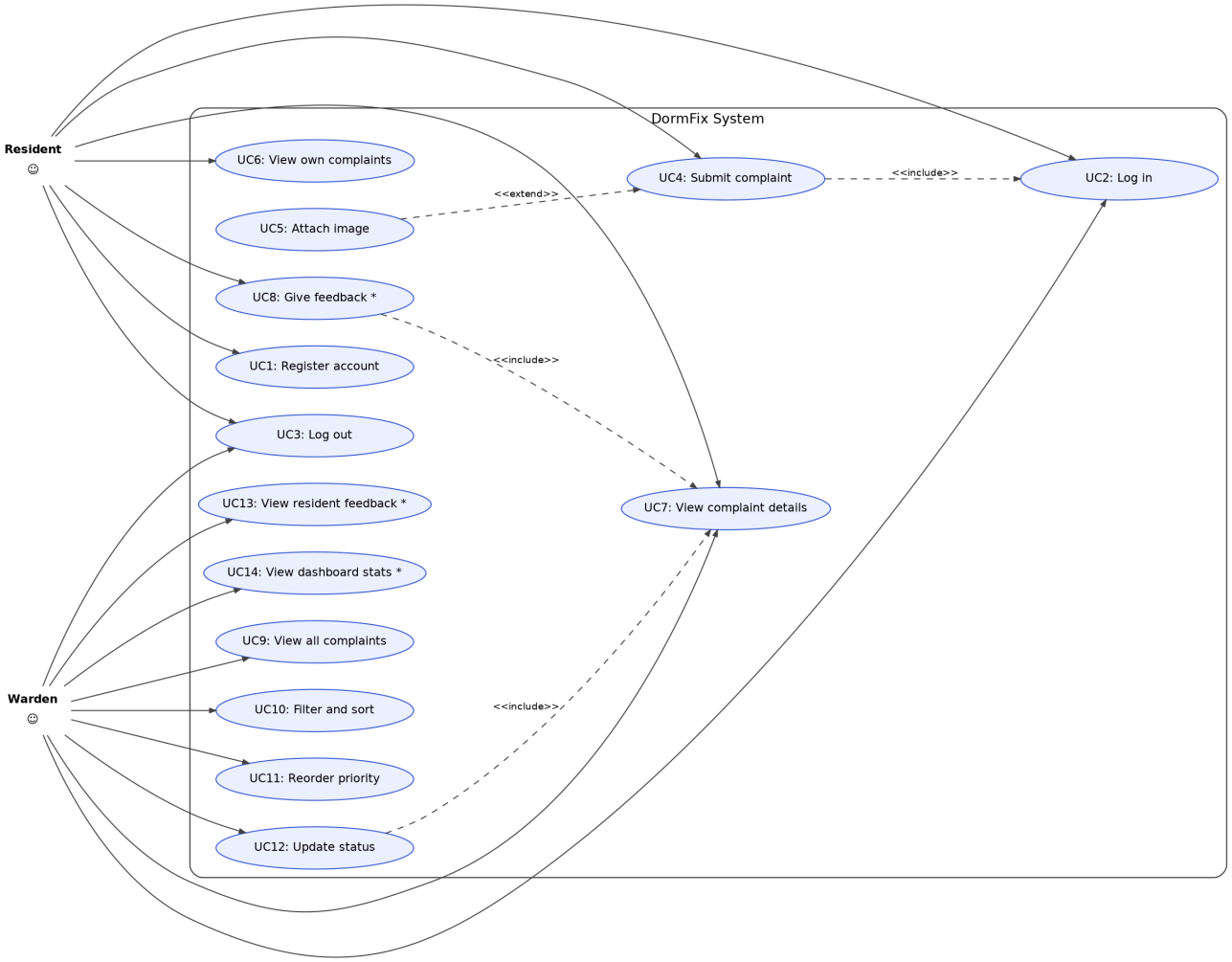


Image rendered with Graphviz. Source: `diagrams/src/01_1_use_case_diagram.dot`. Items marked `*` are implemented in the code but their routes are currently disabled in `server.js` (see Section 9).

Use case descriptions (short form)

ID	Use case	Actor(s)	Backend endpoint	Status in code
UC1	Register account	Resident	`POST /api/auth/register`	Implemented
UC2	Log in	Resident, Warden	`POST /api/auth/login`	Implemented
UC3	Log out	Resident, Warden	(client only, clears localStorage)	Implemented
UC4	Submit complaint	Resident	`POST /api/complaints`	Implemented
UC5	Attach image to complaint	Resident	`POST /api/complaints` (multipart field)	Implemented
UC6	View own complaints	Resident	`GET /api/complaints/my`	Implemented
UC7	View complaint details	Resident, Warden	`GET /api/complaints/:id`	Implemented

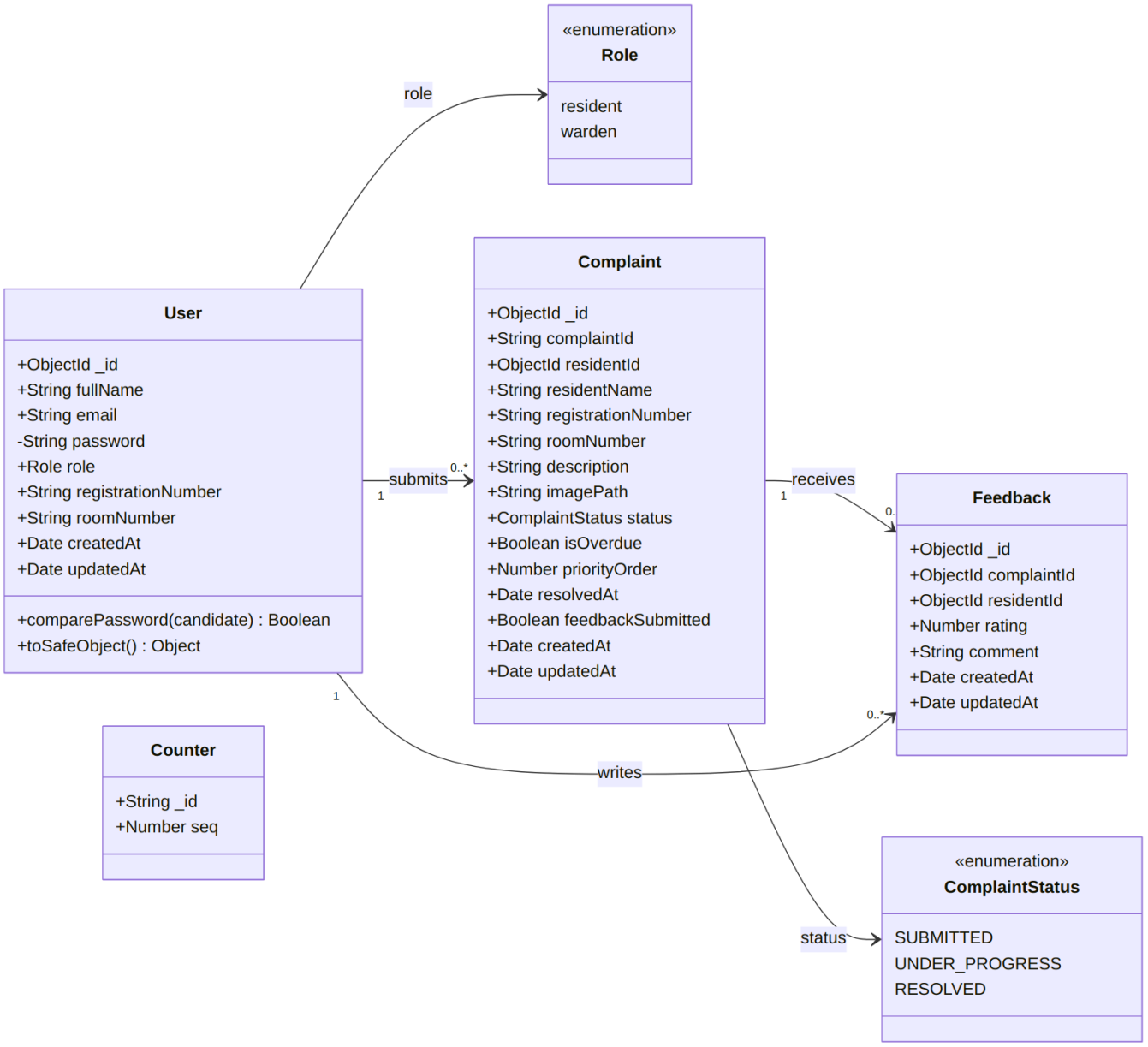
ID	Use case	Actor(s)	Backend endpoint	Status in code
UC8	Give feedback after resolution	Resident	`POST /api/complaints/:id/feedback`	Route disabled (see Section 9)
UC9	View all complaints	Warden	`GET /api/complaints`	Implemented
UC10	Filter and sort complaints	Warden	`GET /api/complaints?status&overdue&...`	Implemented
UC11	Reorder priority up or down	Warden	`PATCH /api/complaints/reorder`	Implemented
UC12	Update complaint status	Warden	`PATCH /api/complaints/:id/status`	Implemented
UC13	View resident feedback	Warden	`GET /api/complaints/:id/feedback`	Route disabled (see Section 9)
UC14	View warden dashboard stats	Warden	`GET /api/dashboard/warden`	Route disabled (see Section 9)

2. Class Diagram

2.1 Domain Model (Persistent Entities)

This diagram shows the four Mongoose models in `server/models/`. Relationships match the `ref` fields and the one-to-one or one-to-many rules enforced in services.





Model constraints worth noting in the report

- `User.password` has `select: false`, so it is excluded from queries by default.
- `User.registrationNumber` is sparse and unique, so wardens have no value for it.
- `Complaint.complaintId` is unique and generated from `Counter` as `HF-0001`, `HF-0002`, and so on.
- `Feedback.complaintId` is unique, which enforces one feedback per complaint.
- `Feedback.rating` is restricted to the range 1 to 5.

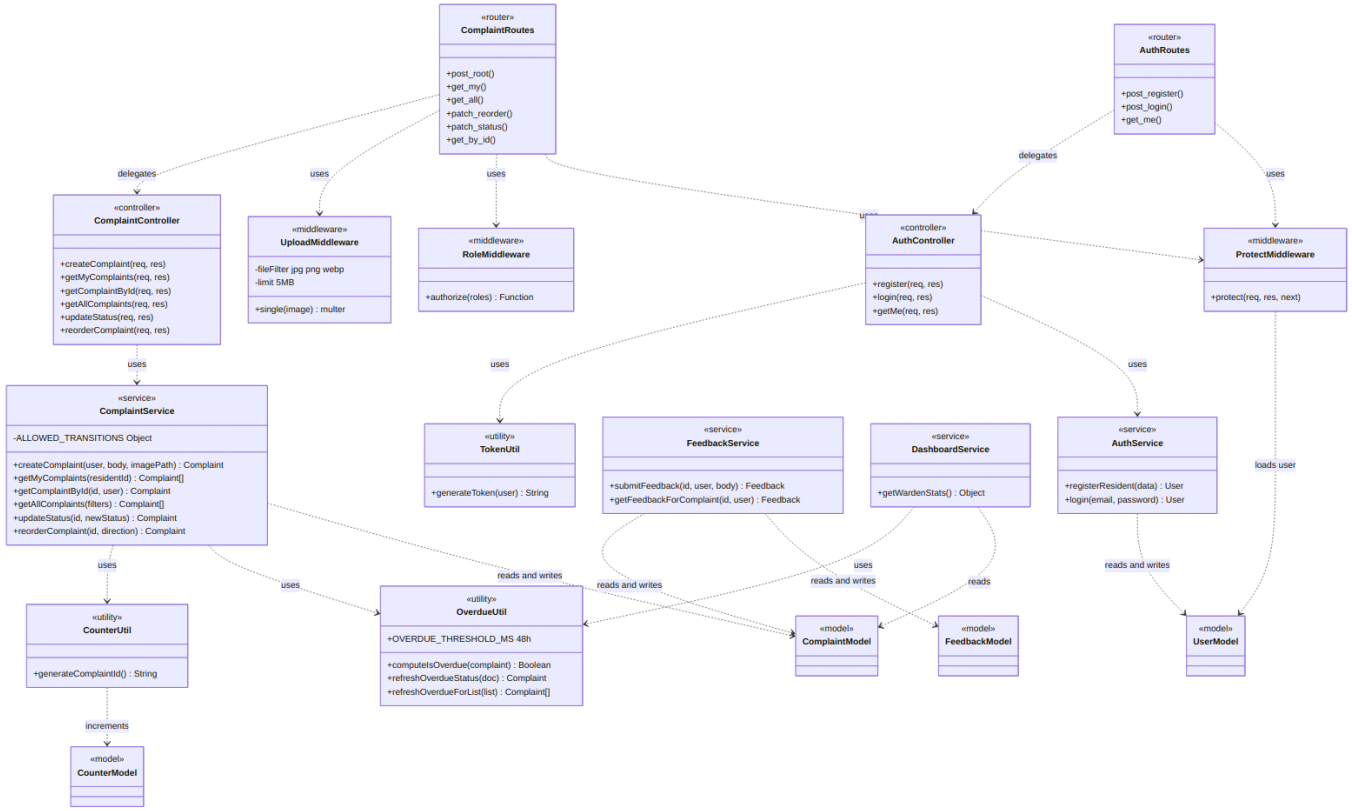
2.2 Backend Layered Class Structure

This diagram shows the layered architecture: Routes call Middleware, then Controllers, then Services, which in turn use Models and Utils.

Syntax error in text

mermaid version 12.2.0

Parse error on line 139: ...rModel : loads user</model></model></mo
----------------------^ Expecting 'EOF', 'acc_title', 'acc_descr',
'acc_descr_multiline_value', 'NAMESPACE', 'ANNOTATION_START', 'CLASS',
'MEMBER', 'SEPARATOR', 'NOTE_FOR', 'NOTE', 'CLASSDEF', 'ALPHA',...



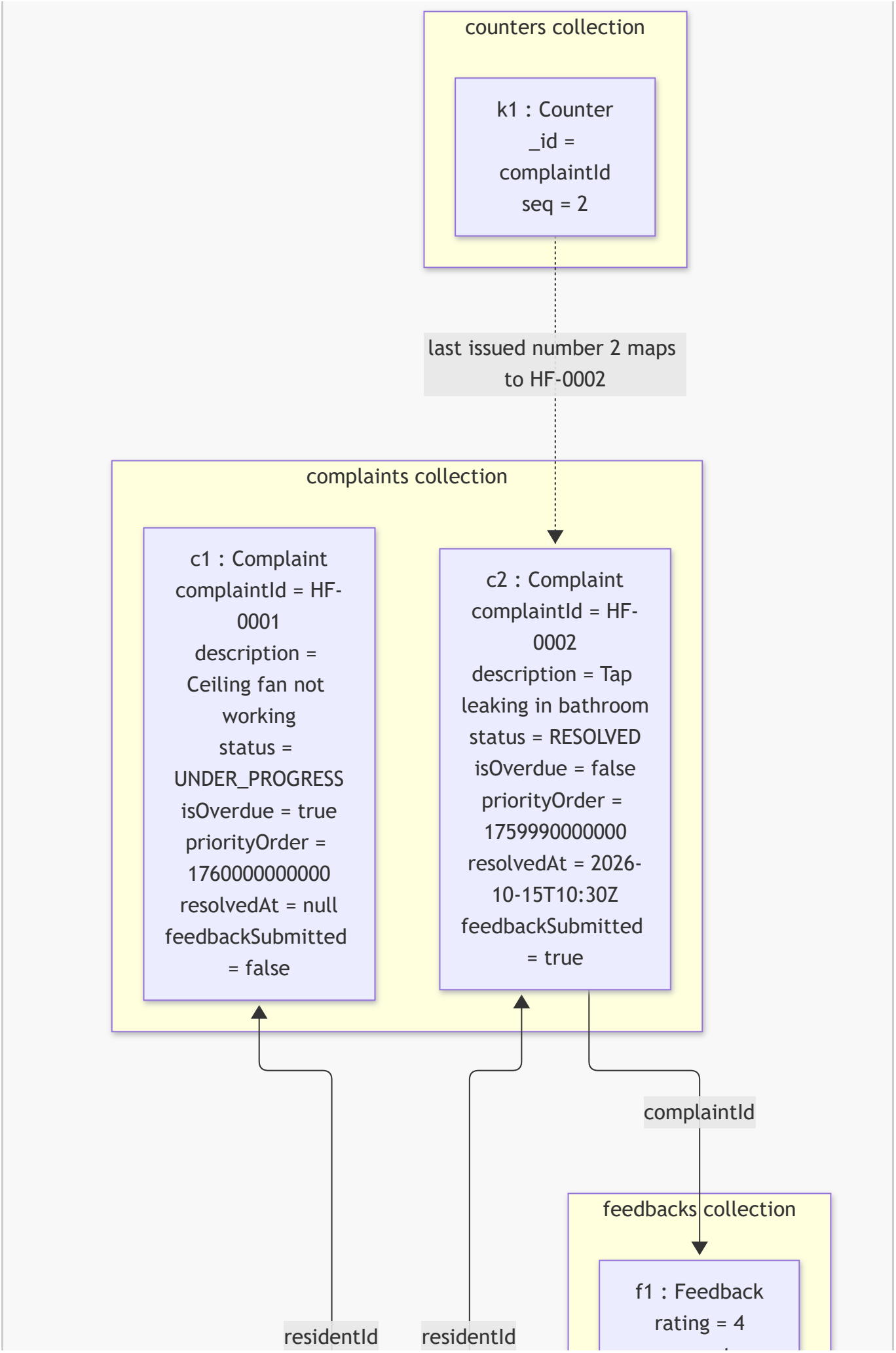
Design notes

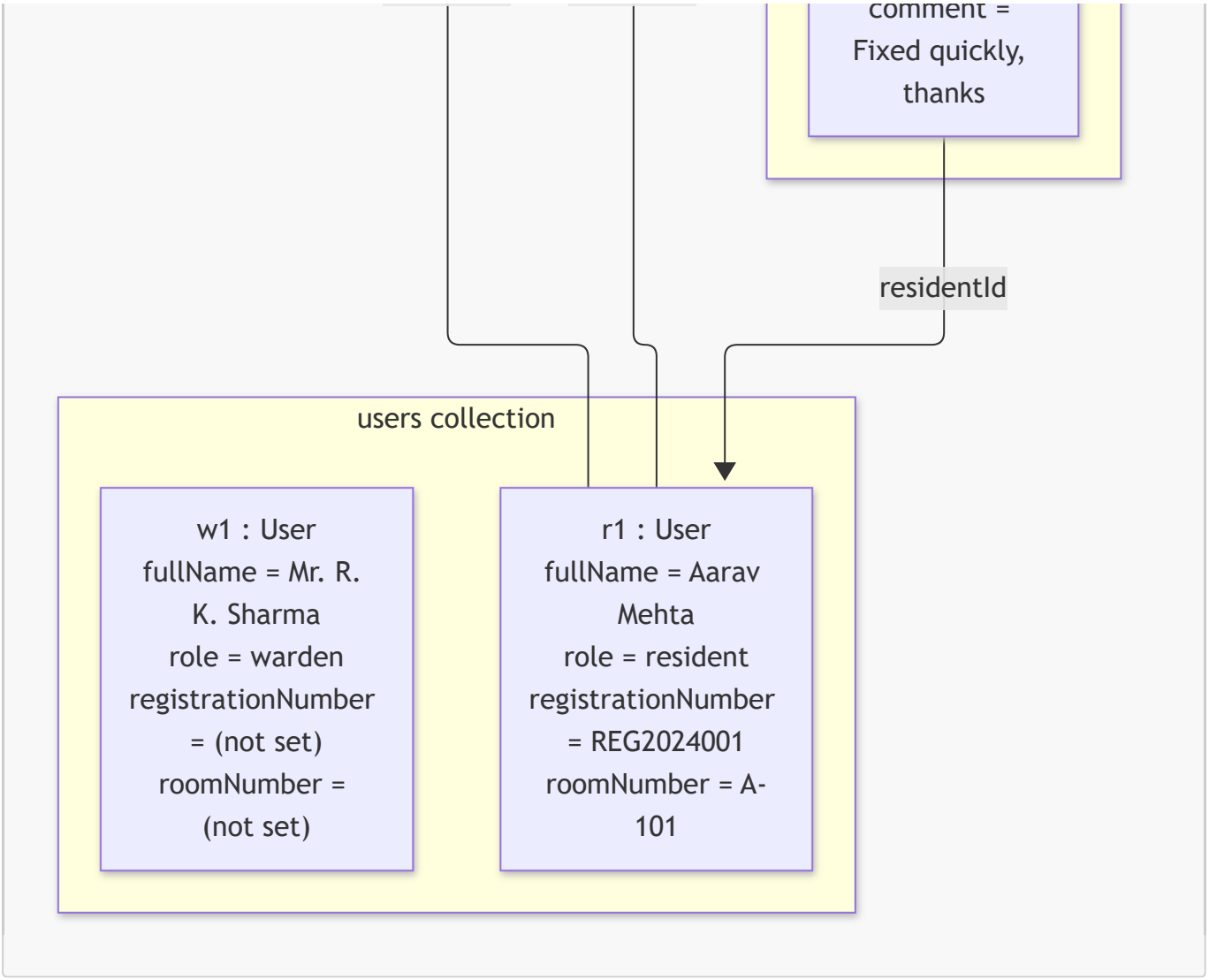
- Controllers contain no business logic. They only read `req`, call a service, and format the response with `success()` or `error()`.
- Services throw an `AppError` with an HTTP status code. The global `errorHandler` converts it into the response.
- `ComplaintService` checks ownership for residents itself, because the route has no per-object check.

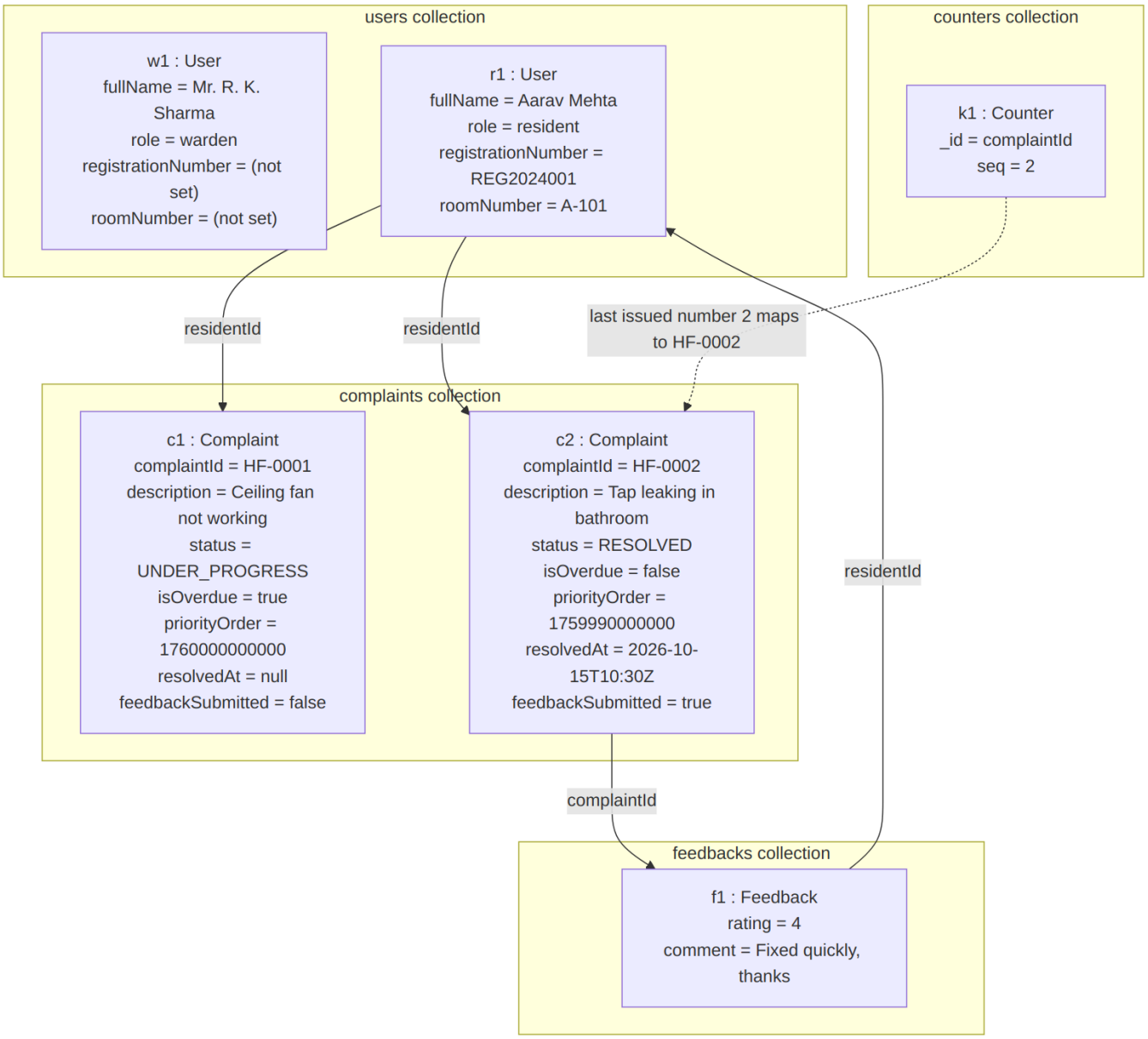
---

3. Object Diagram

This is a runtime snapshot of the database after a sequence of actions: two residents exist, one complaint is in progress and another is resolved with feedback. Values are illustrative and based on the seed data in `server/seed/seed.js`. The complaint `HF-0002` is the case in which the resident has submitted feedback.


counters collection
k1 : Counter
_id = complaintId
seq = 2
last issued number 2 maps to HF-0002
complaints collection
c1 : Complaint
complaintId = HF-0001
description = Ceiling fan not working
status = UNDER_PROGRESS
isOverdue = true
priorityOrder = 1760000000000
resolvedAt = null
feedbackSubmitted = false
c2 : Complaint
complaintId = HF-0002
description = Tap leaking in bathroom
status = RESOLVED
isOverdue = false
priorityOrder = 1759990000000
resolvedAt = 2026-10-15T10:30Z
feedbackSubmitted = true
residentId
residentId
complaintId
feedbacks collection
f1 : Feedback
rating = 4


comment =
Fixed quickly,
thanks
residentId
users collection
w1 : User
fullName = Mr. R.
K. Sharma
role = warden
registrationNumber
= (not set)
roomNumber =
(not set)
r1 : User
fullName = Aarav
Mehta
role = resident
registrationNumber
= REG2024001
roomNumber = A-
101

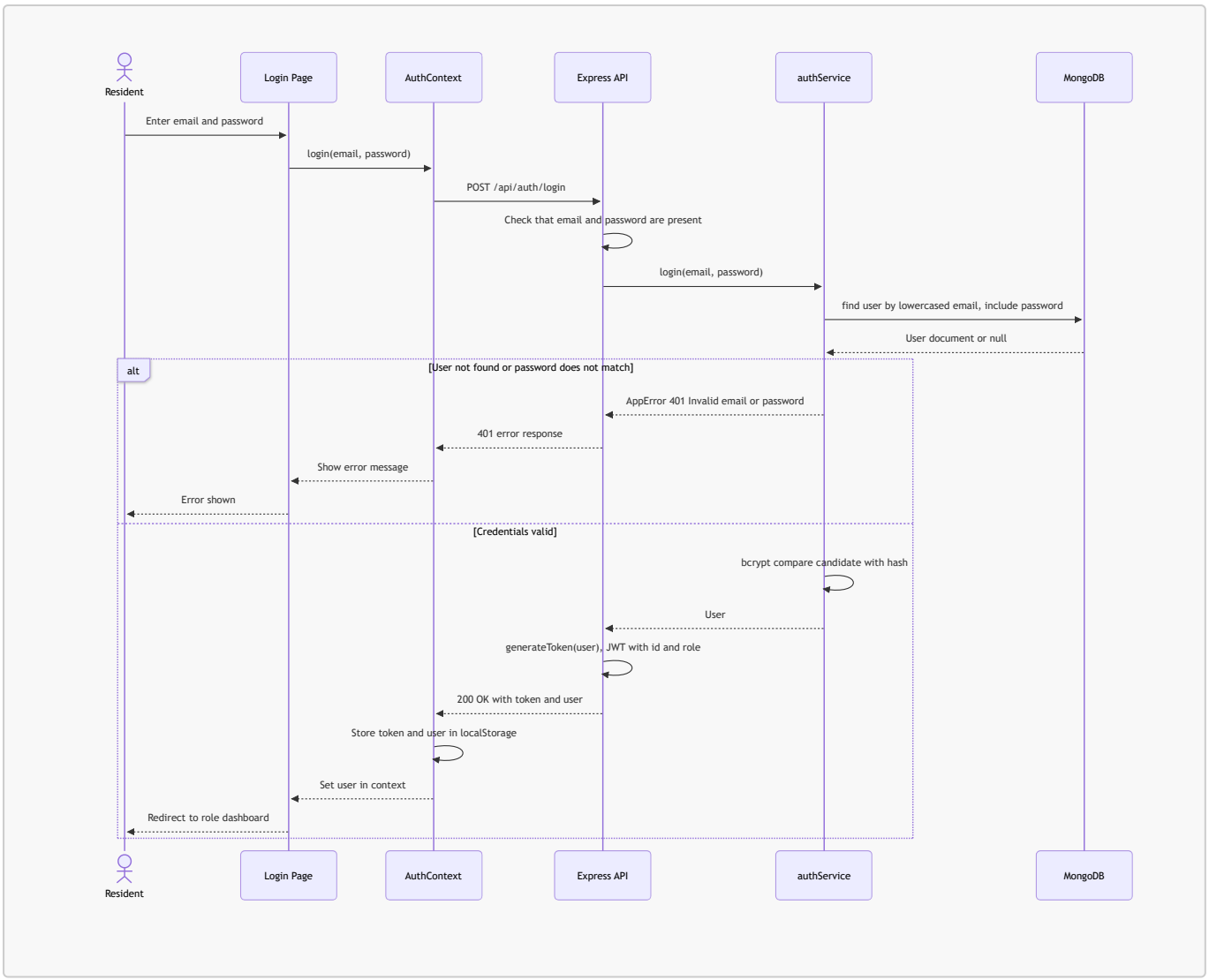


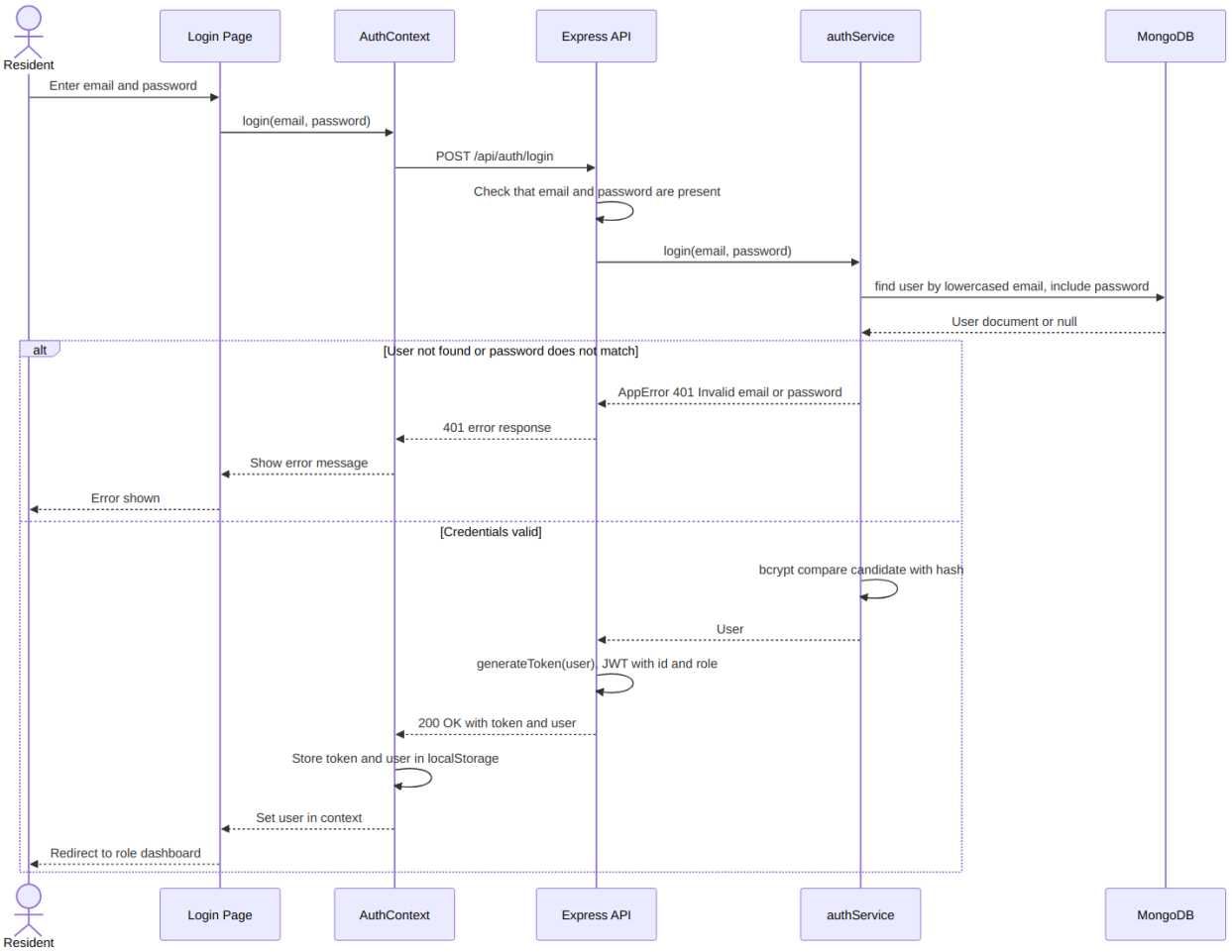
Reading the snapshot: `c1` is flagged `isOverdue = true` because its `createdAt` is more than 48 hours old and it is not yet resolved. `c2` is not overdue because it is resolved. `k1.seq = 2` means the next complaint will receive the ID `HF-0003`.

---

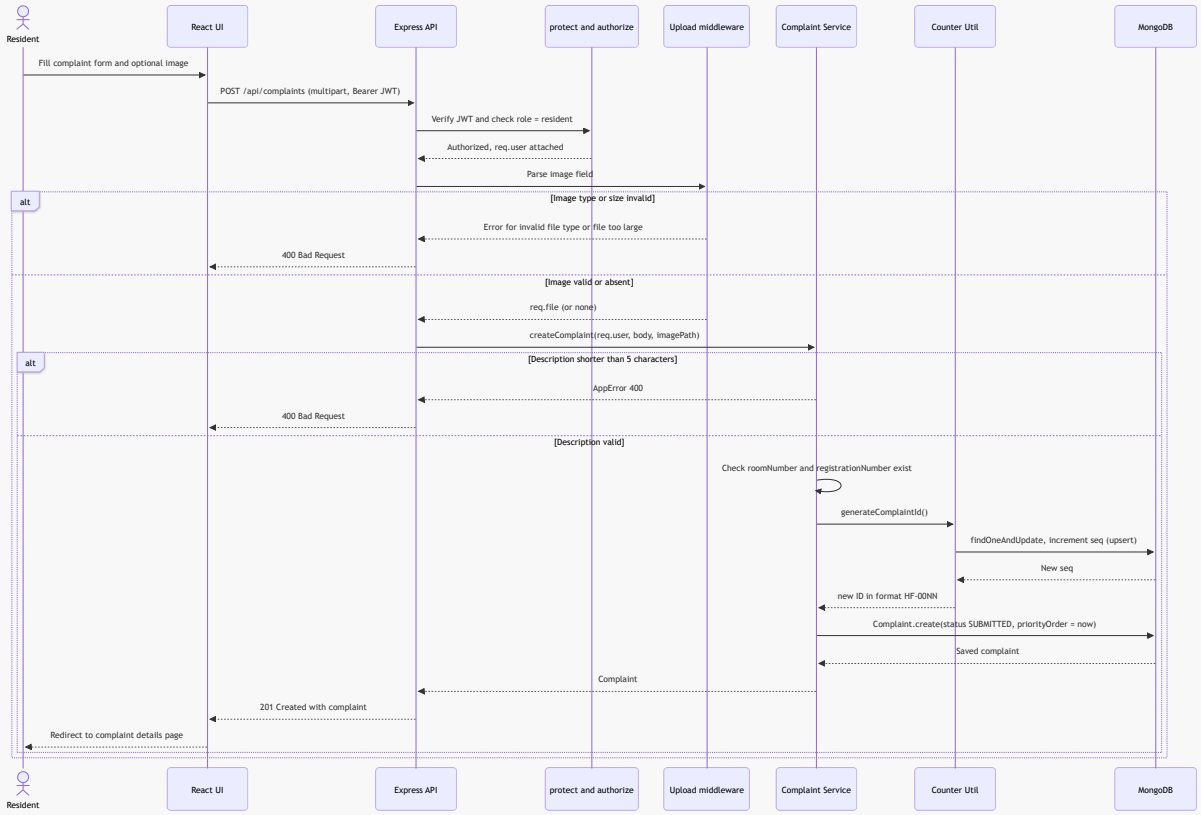
4. Sequence Diagrams

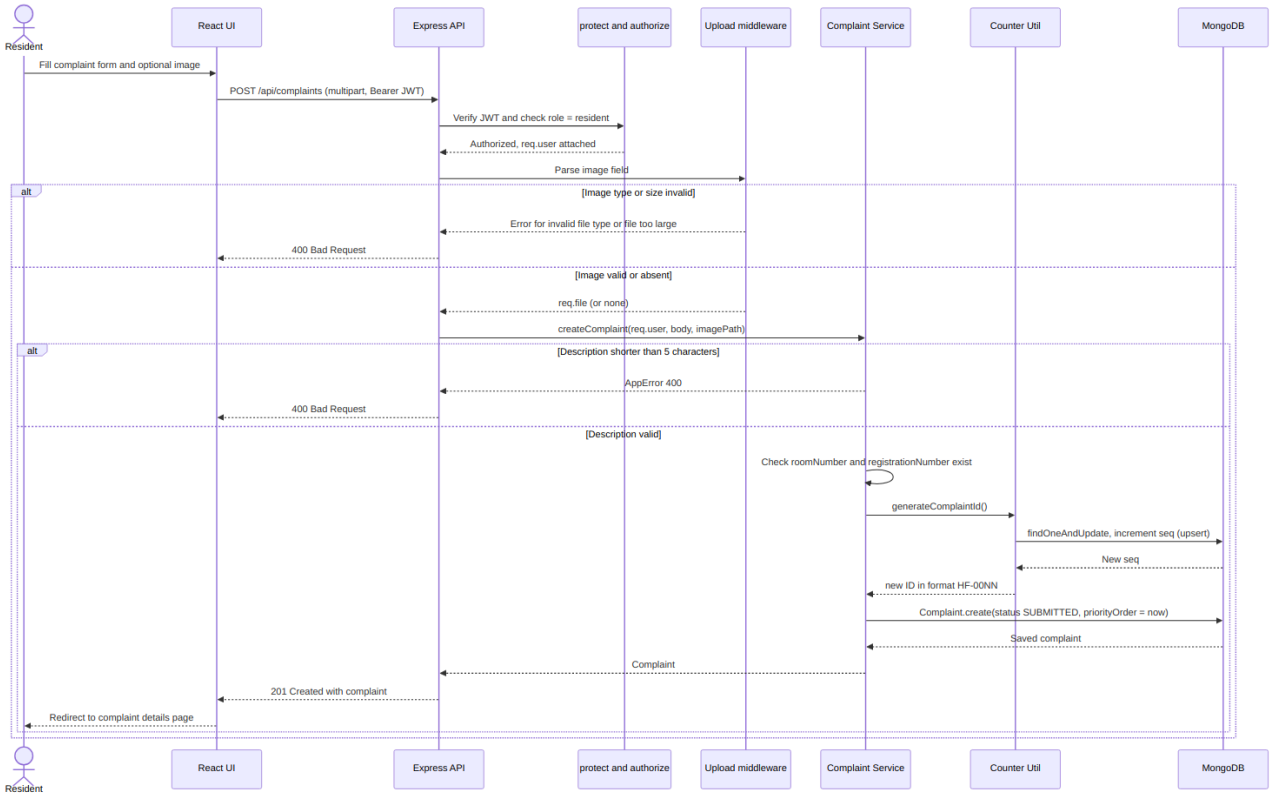
4.1 Resident Login


Resident
Login Page
AuthContext
Express API
authService
MongoDB
Enter email and password
login(email, password)
POST /api/auth/login
Check that email and password are present
login(email, password)
find user by lowercased email, include password
User document or null
alt
[User not found or password does not match]
AppError 401 Invalid email or password
401 error response
Show error message
Error shown
[Credentials valid]
bcrypt compare candidate with hash
User
generateToken(user), JWT with id and role
200 OK with token and user
Store token and user in localStorage
Set user in context
Redirect to role dashboard
Resident
Login Page
AuthContext
Express API
authService
MongoDB

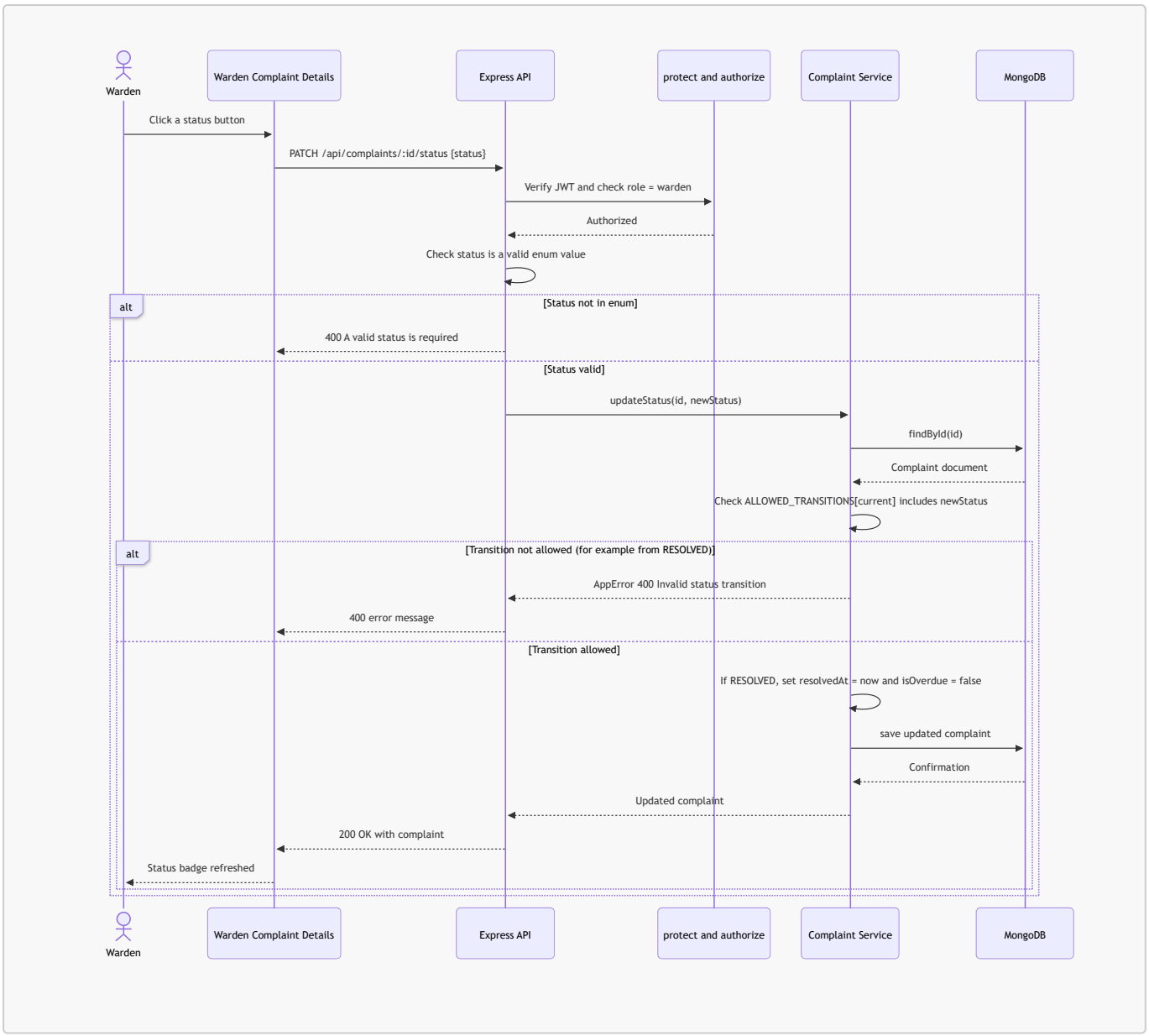


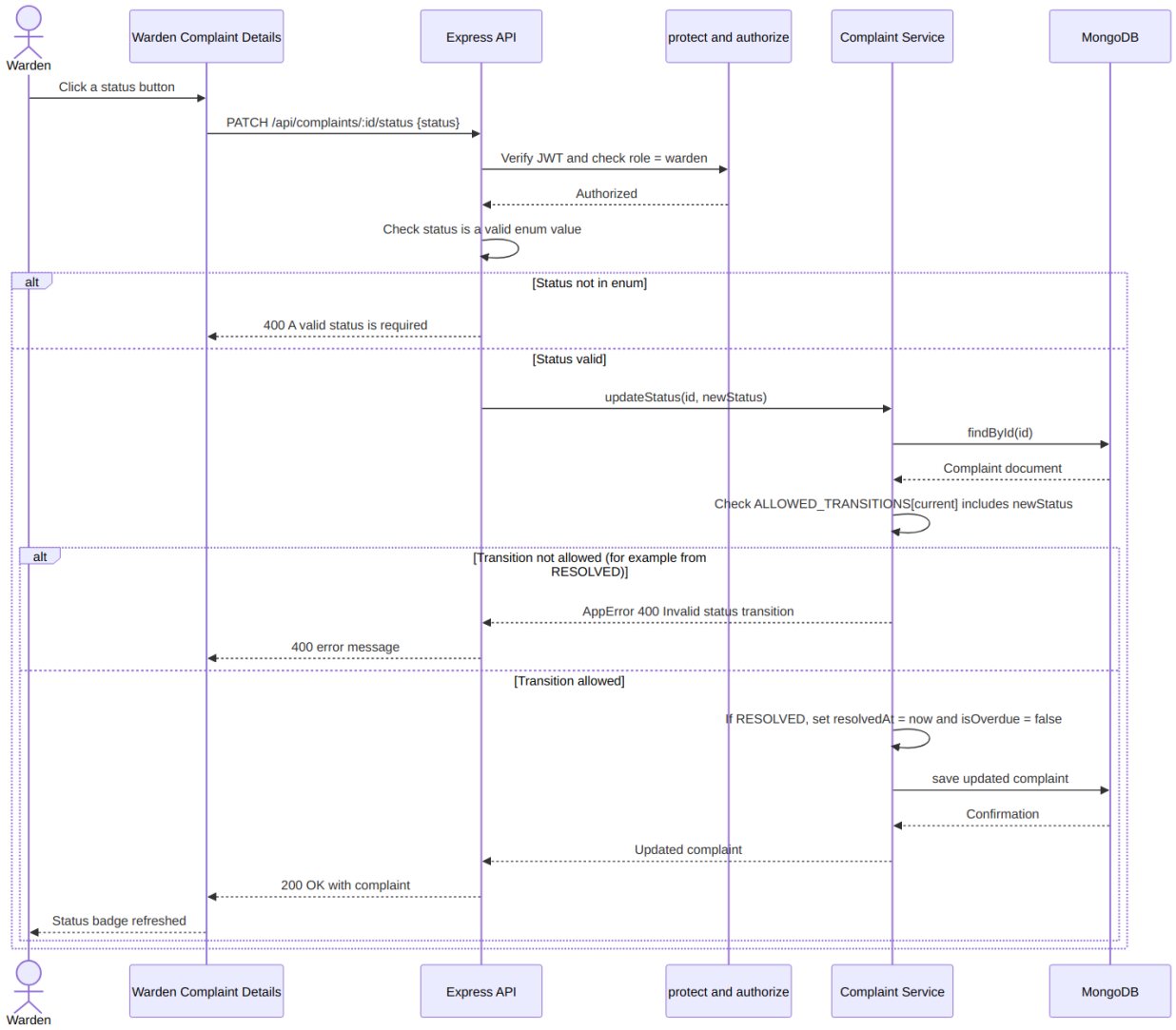
4.2 Resident Submits Complaint



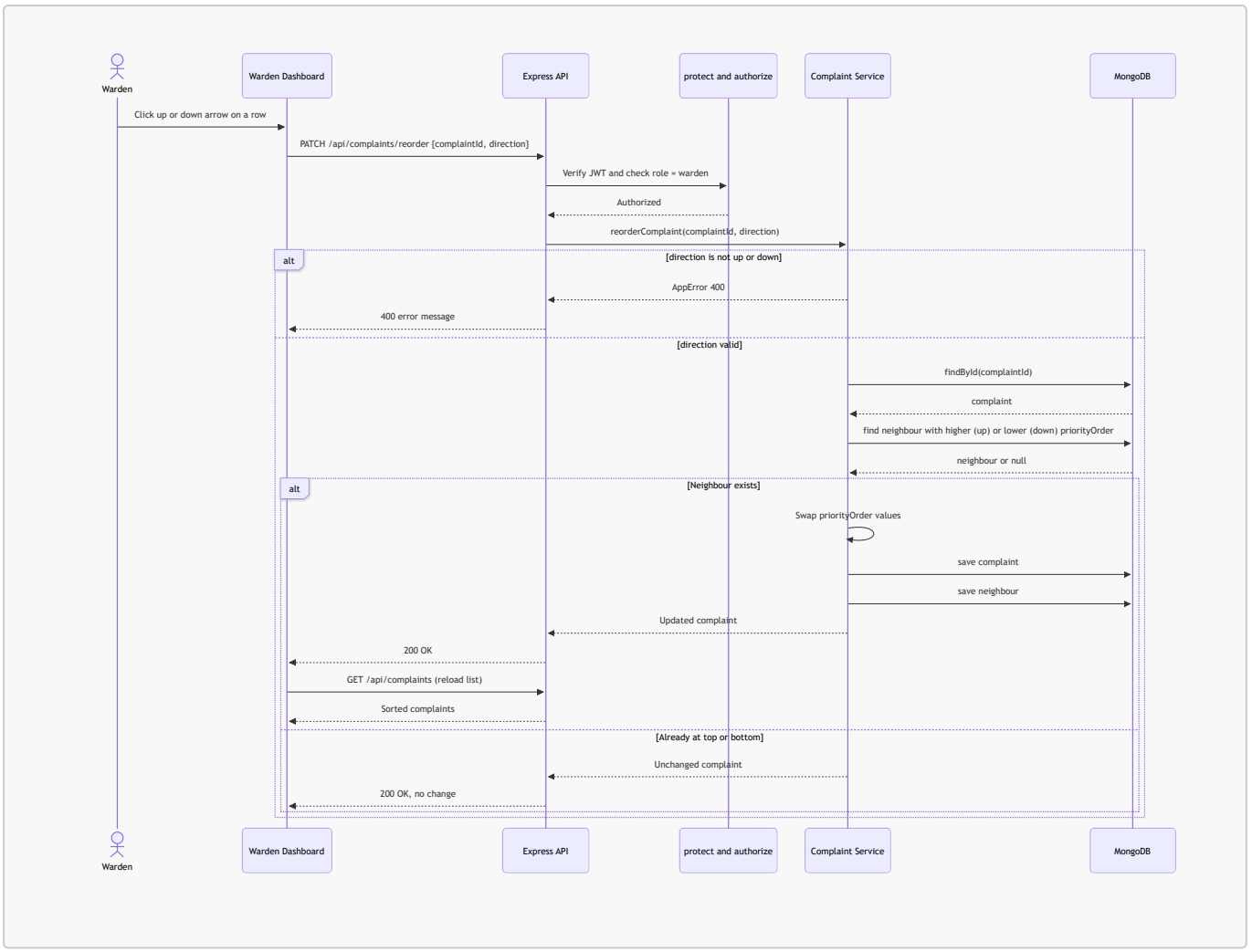


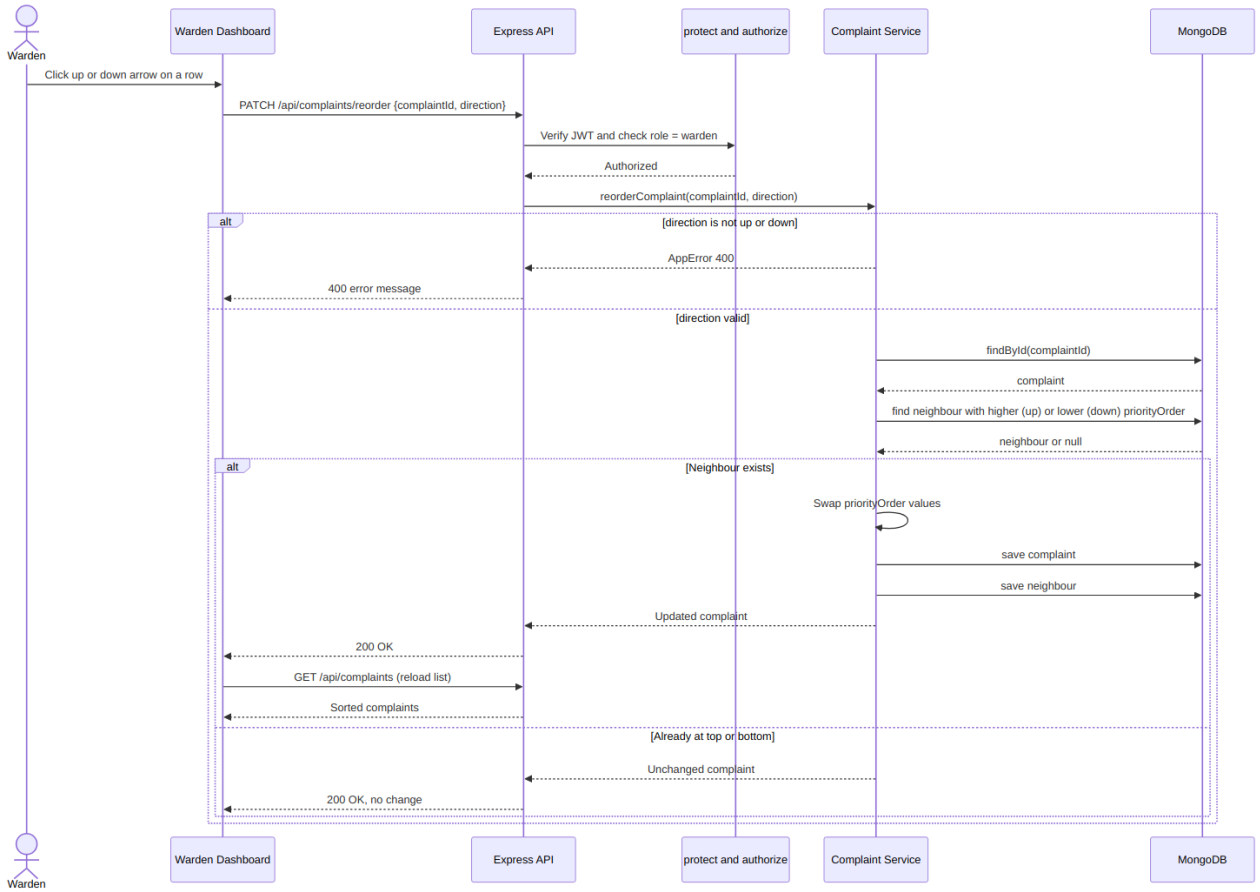
4.3 Warden Updates Complaint Status


Warden
Warden Complaint Details
Express API
protect and authorize
Complaint Service
MongoDB
Click a status button
PATCH /api/complaints/:id/status {status}
Verify JWT and check role = warden
Authorized
Check status is a valid enum value
alt
[Status not in enum]
400 A valid status is required
[Status valid]
updateStatus(id, newStatus)
findById(id)
Complaint document
Check ALLOWED_TRANSITIONS[current] includes newStatus
alt
[Transition not allowed (for example from RESOLVED)]
AppError 400 Invalid status transition
400 error message
[Transition allowed]
If RESOLVED, set resolvedAt = now and isOverdue = false
save updated complaint
Confirmation
Updated complaint
200 OK with complaint
Status badge refreshed
Warden
Warden Complaint Details
Express API
protect and authorize
Complaint Service
MongoDB

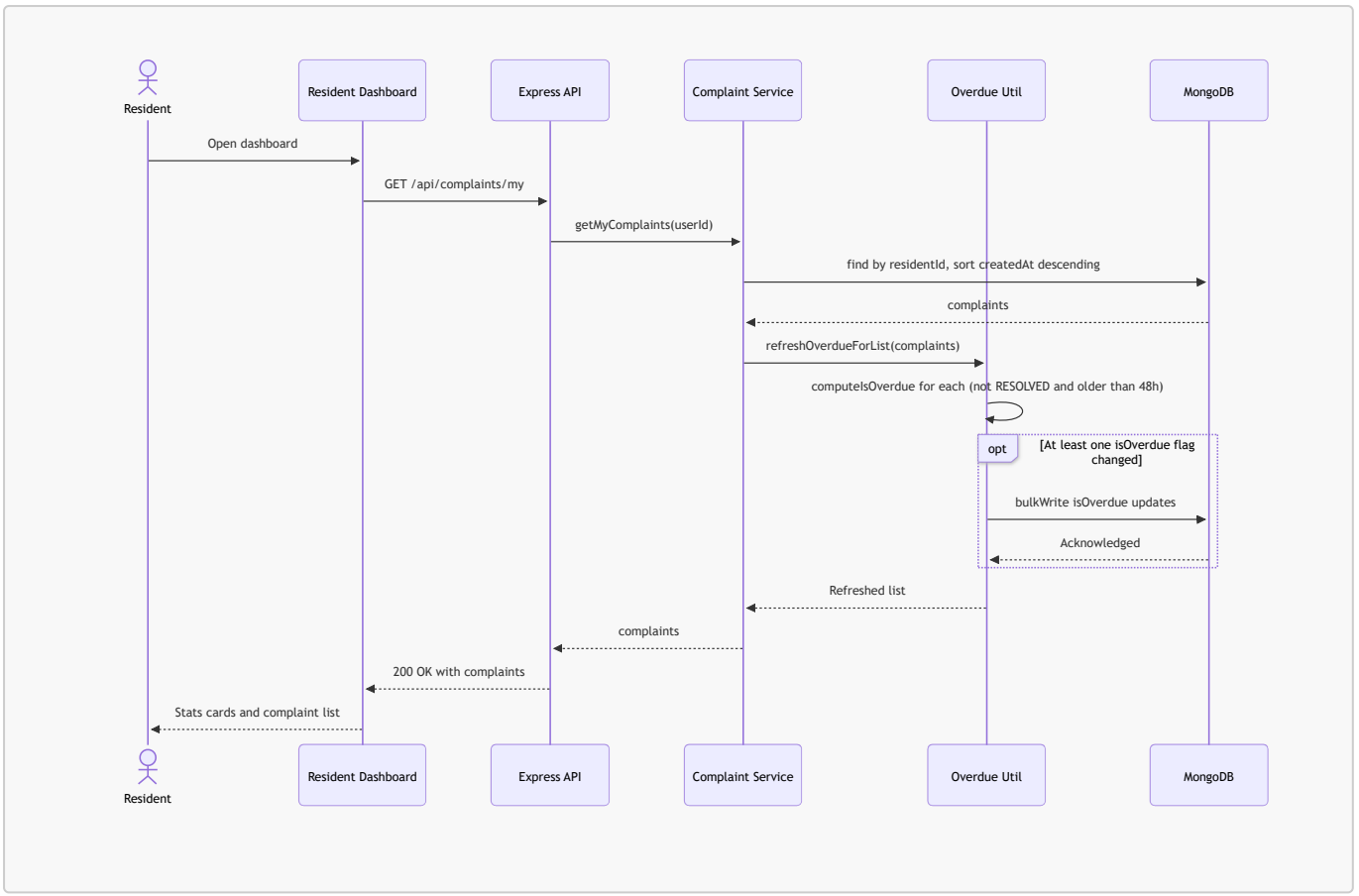


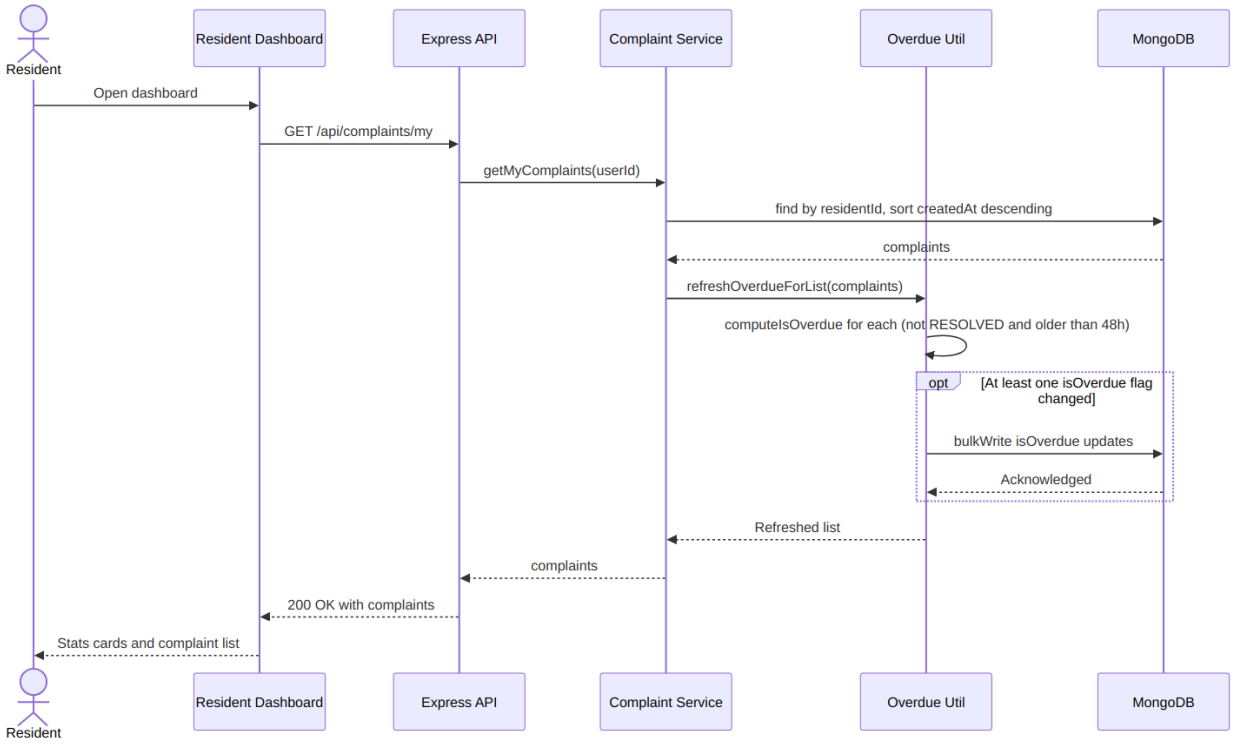
4.4 Warden Reorders Complaint Priority





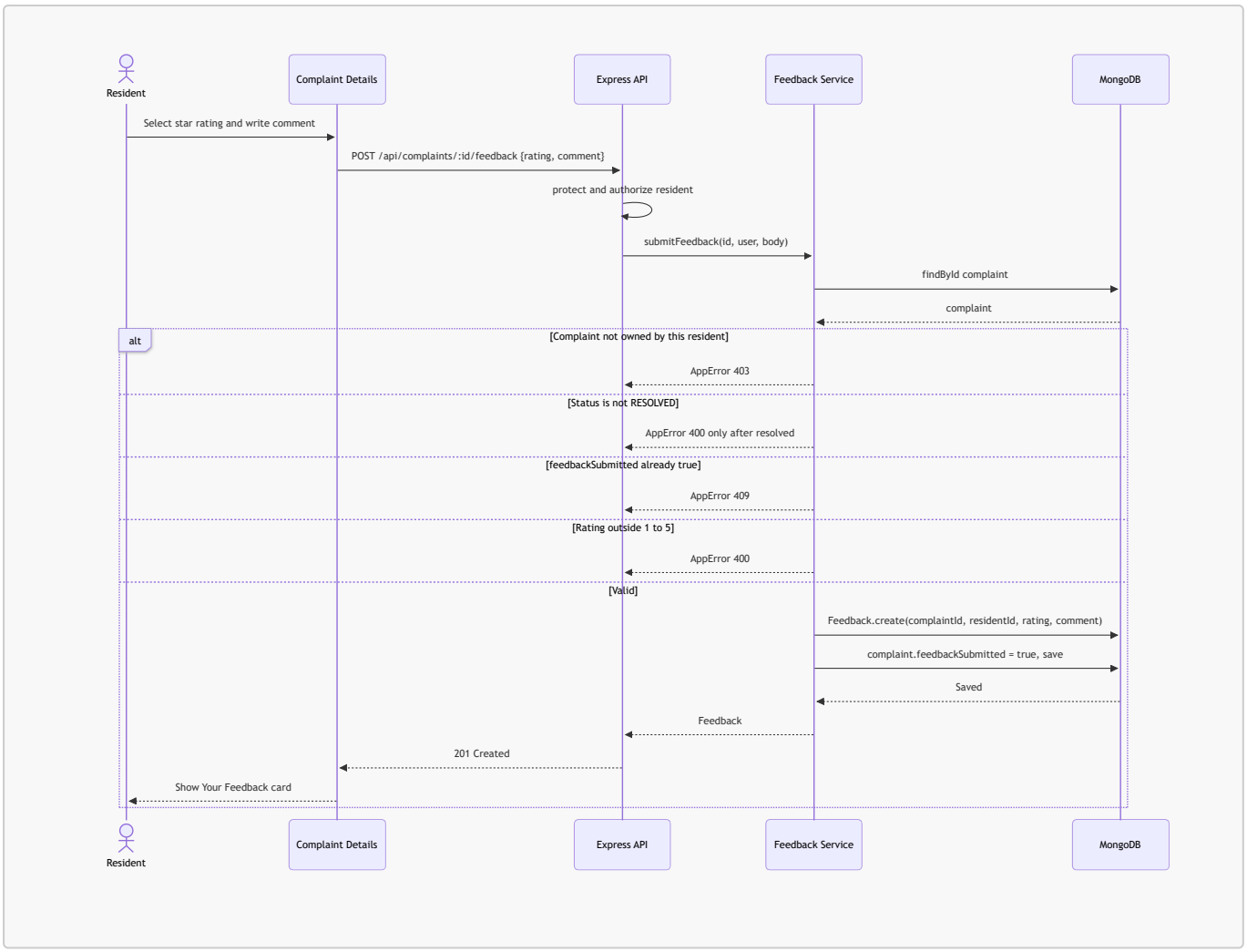
4.5 Resident Views Own Complaints (Overdue Refresh)

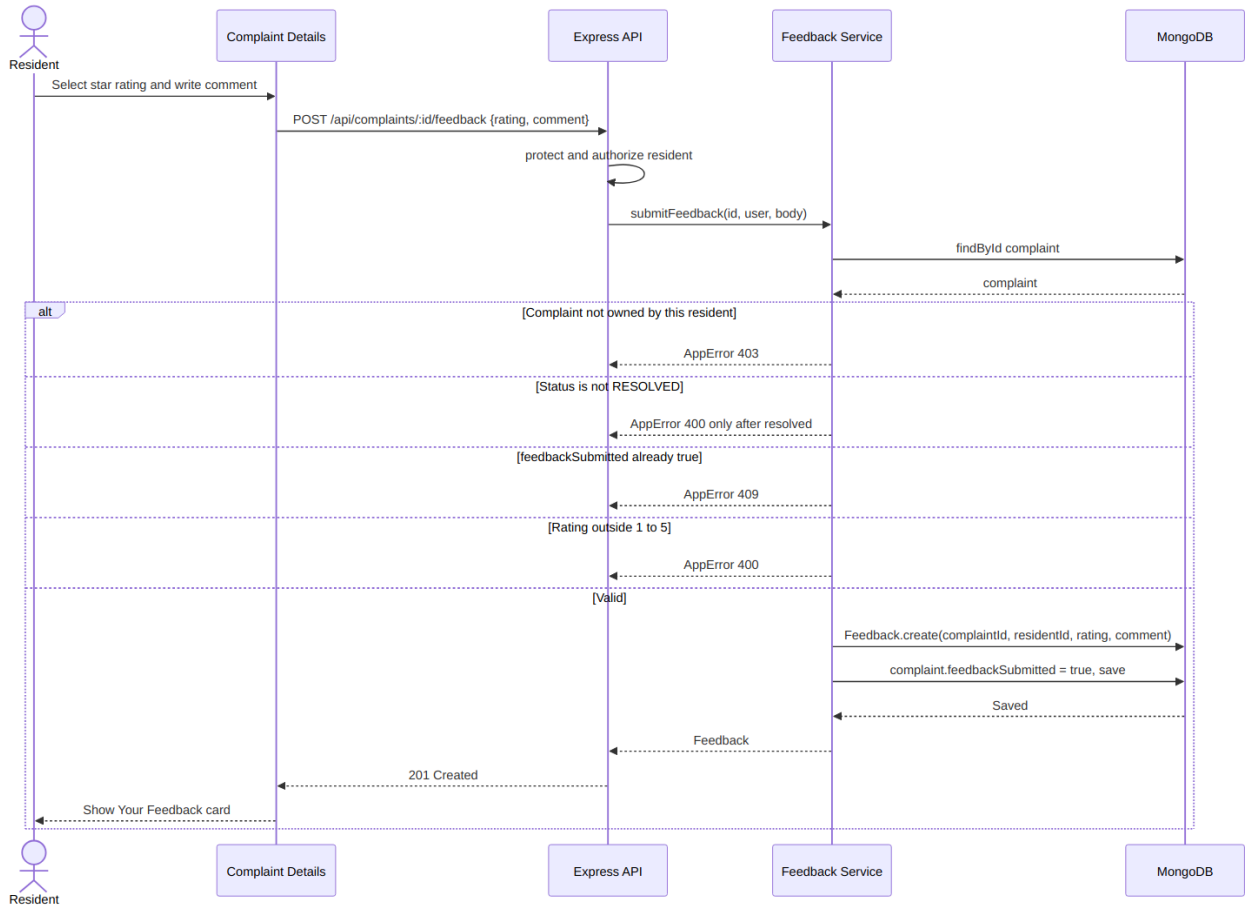




4.6 Resident Submits Feedback (Designed Flow)

> This flow is implemented in `feedbackController.js` and `feedbackService.js`, and the client calls it from `ComplaintDetails.jsx`. However, the route is currently commented out in `server/server.js`, so in the current build this request receives a 404. The diagram shows the intended behaviour.

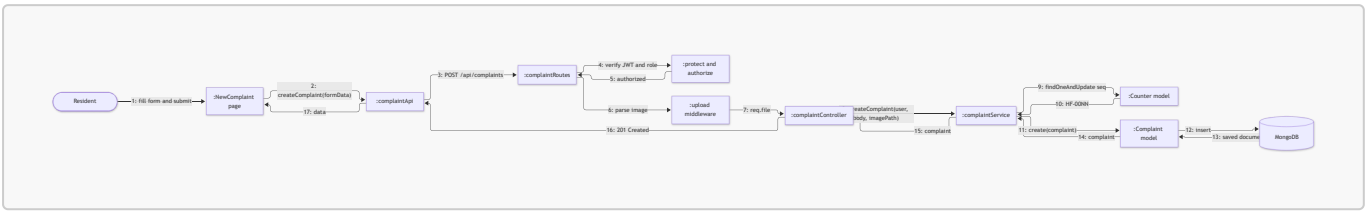

Resident
Complaint Details
Express API
Feedback Service
MongoDB
Select star rating and write comment
POST /api/complaints/:id/feedback {rating, comment}
protect and authorize resident
submitFeedback(id, user, body)
findById complaint
complaint
alt
[Complaint not owned by this resident]
AppError 403
[Status is not RESOLVED]
AppError 400 only after resolved
[feedbackSubmitted already true]
AppError 409
[Rating outside 1 to 5]
AppError 400
[Valid]
Feedback.create(complaintId, residentId, rating, comment)
complaint.feedbackSubmitted = true, save
Saved
Feedback
201 Created
Show Your Feedback card
Resident
Complaint Details
Express API
Feedback Service
MongoDB

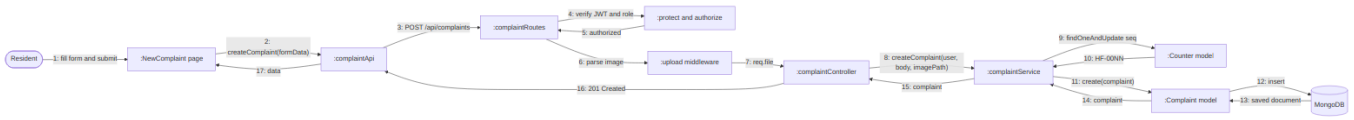

Resident
Complaint Details
Express API
Feedback Service
MongoDB
Select star rating and write comment
POST /api/complaints/:id/feedback {rating, comment}
protect and authorize resident
submitFeedback(id, user, body)
findById complaint
complaint
alt
[Complaint not owned by this resident]
AppError 403
[Status is not RESOLVED]
AppError 400 only after resolved
[feedbackSubmitted already true]
AppError 409
[Rating outside 1 to 5]
AppError 400
[Valid]
Feedback.create(complaintId, residentId, rating, comment)
complaint.feedbackSubmitted = true, save
Saved
Feedback
201 Created
Show Your Feedback card
Resident
Complaint Details
Express API
Feedback Service
MongoDB

5. Collaboration Diagrams

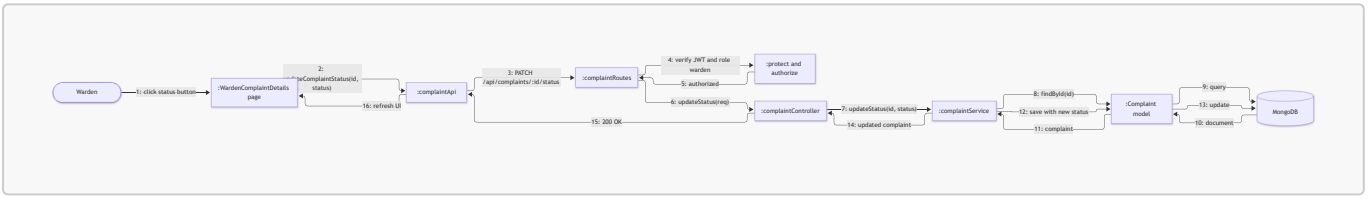
Collaboration diagrams show the same interactions as sequence diagrams, but emphasise the structural links between objects. Messages are numbered in their order of execution.

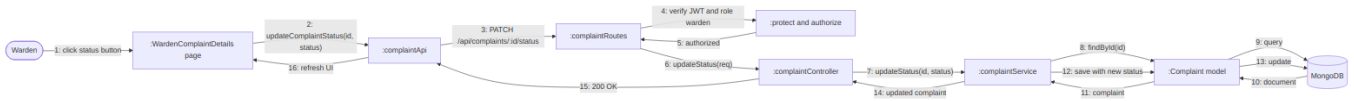
5.1 Collaboration: Submit Complaint





5.2 Collaboration: Warden Updates Status



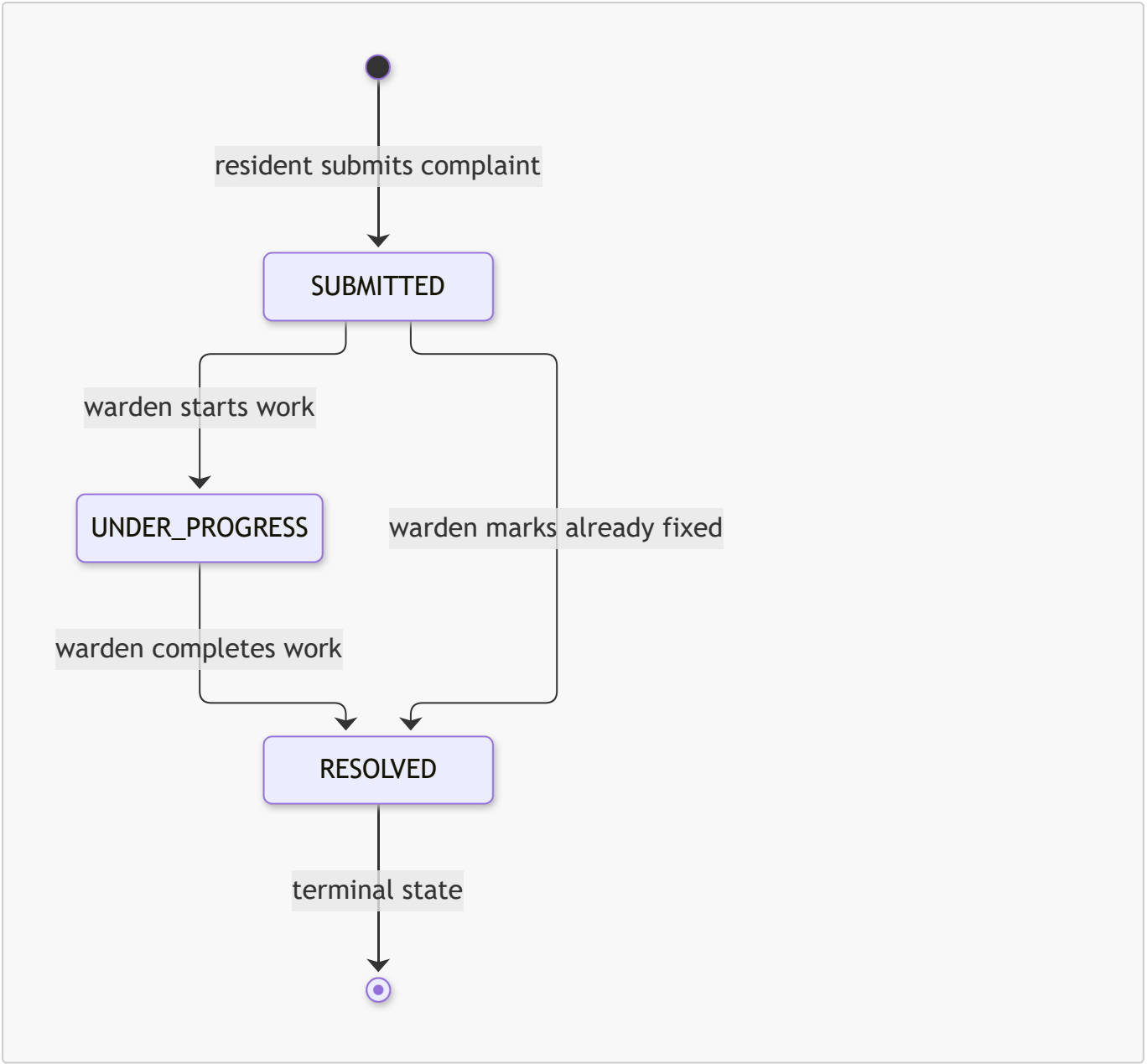


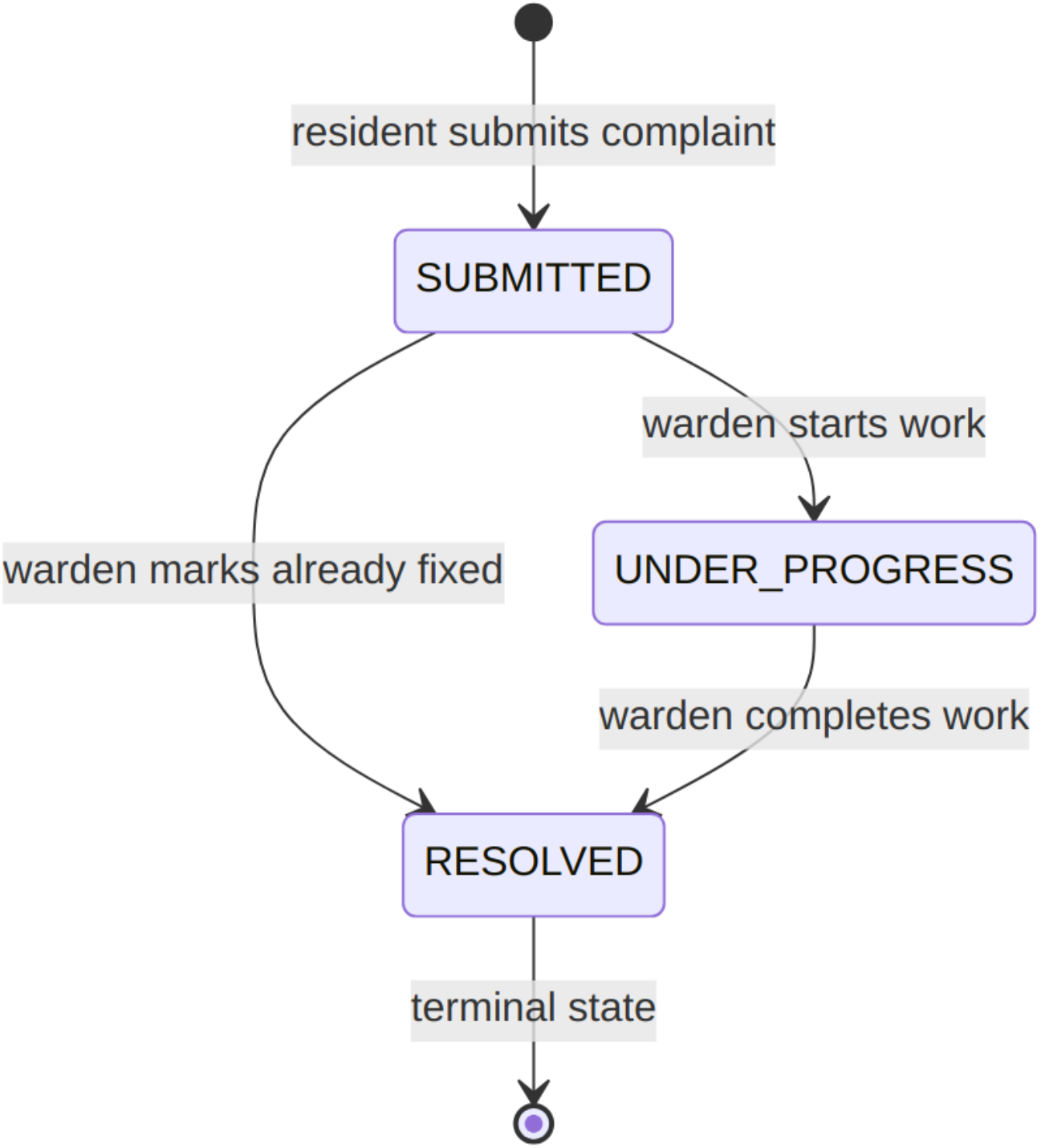
---

6. State Diagrams

6.1 Complaint Lifecycle (status field)

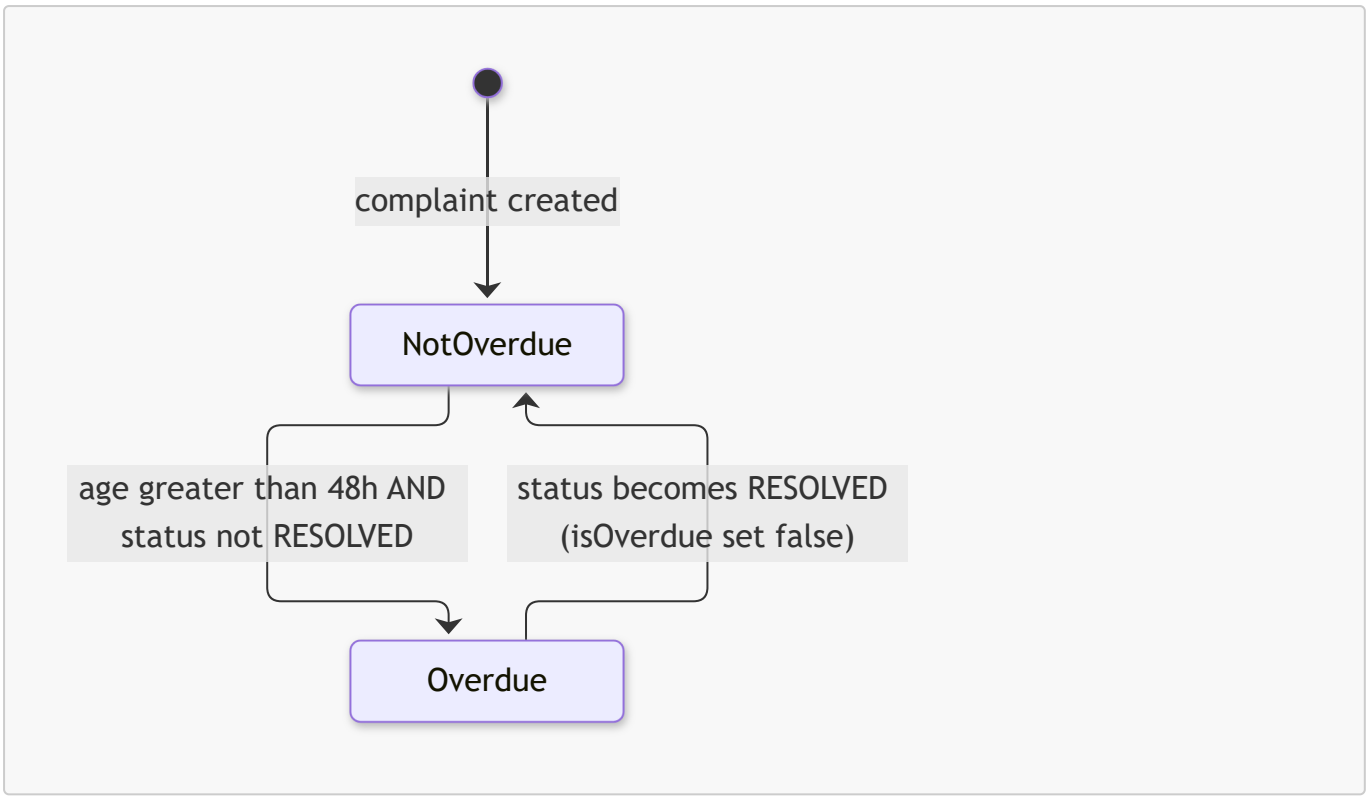
The `status` field of `Complaint` moves only through the transitions defined in `ALLOWED_TRANSITIONS` in `complaintService.js`. `RESOLVED` is terminal.

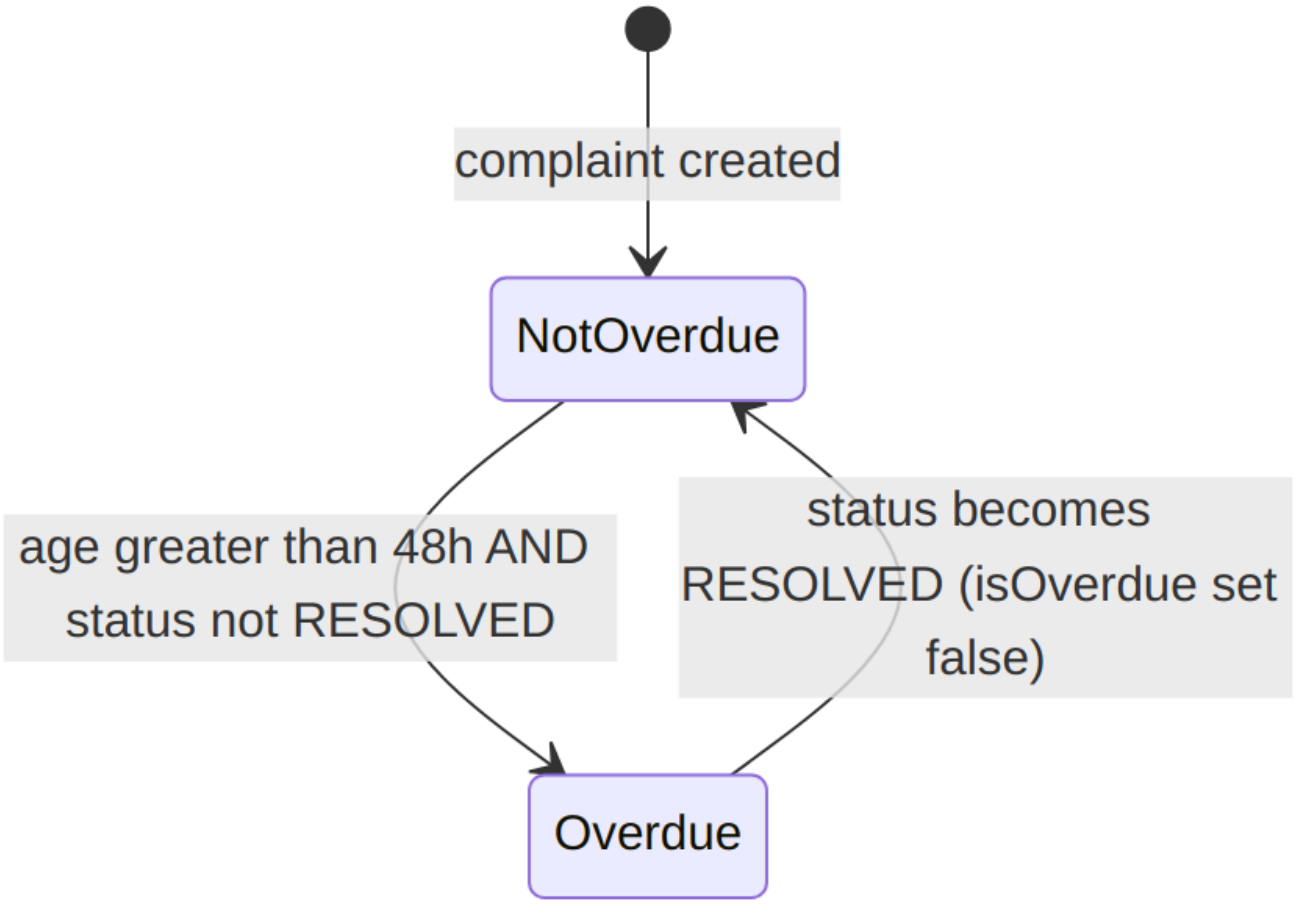

resident submits complaint
SUBMITTED
warden starts work
UNDER_PROGRESS
warden marks already fixed
warden completes work
RESOLVED
terminal state



6.2 Overdue Flag (isOverdue, derived)

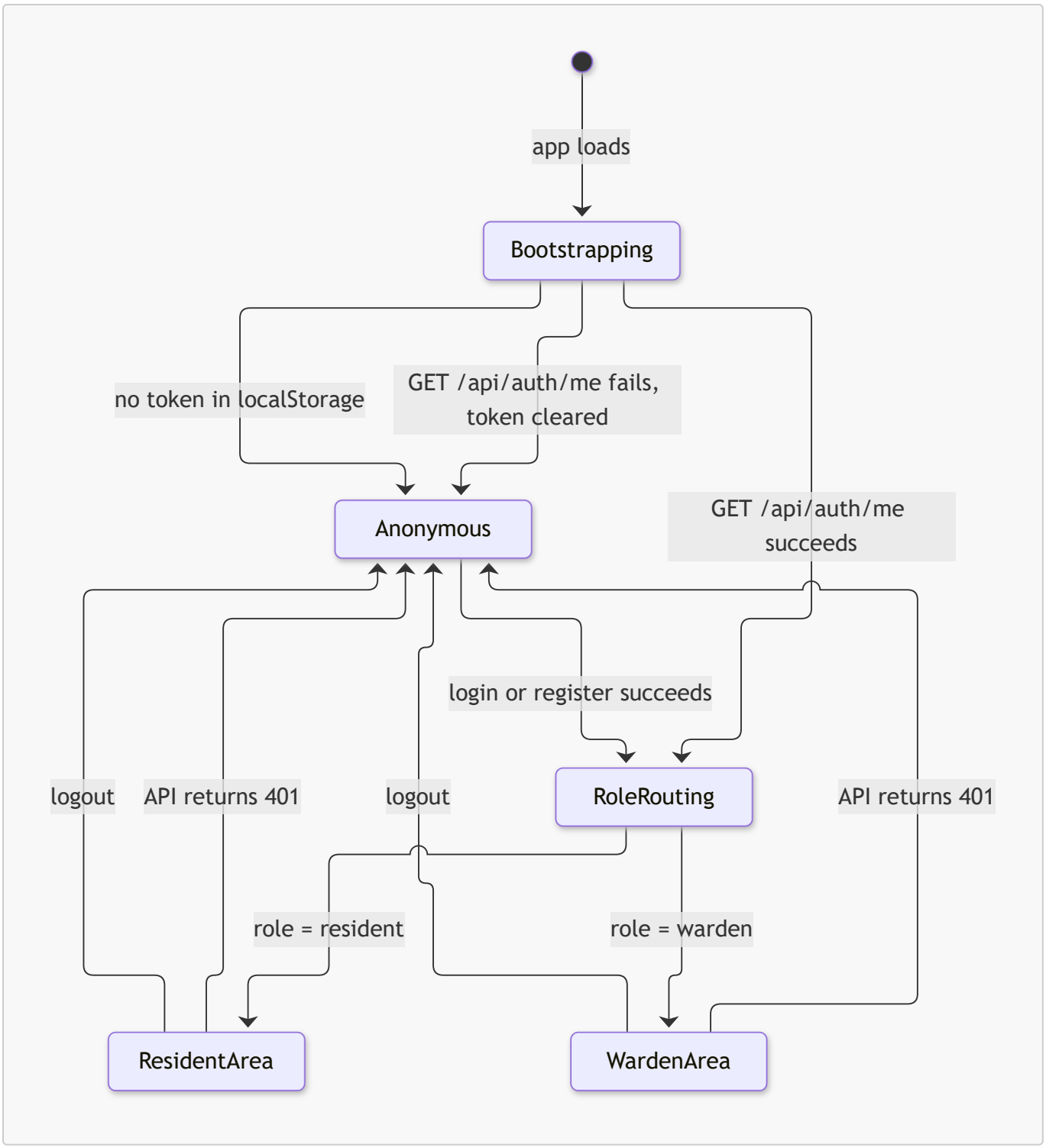
`isOverdue` is not stored as a user input. It is recalculated by the server every time a complaint is read, using `computeIsOverdue()` in `overdue.js`.

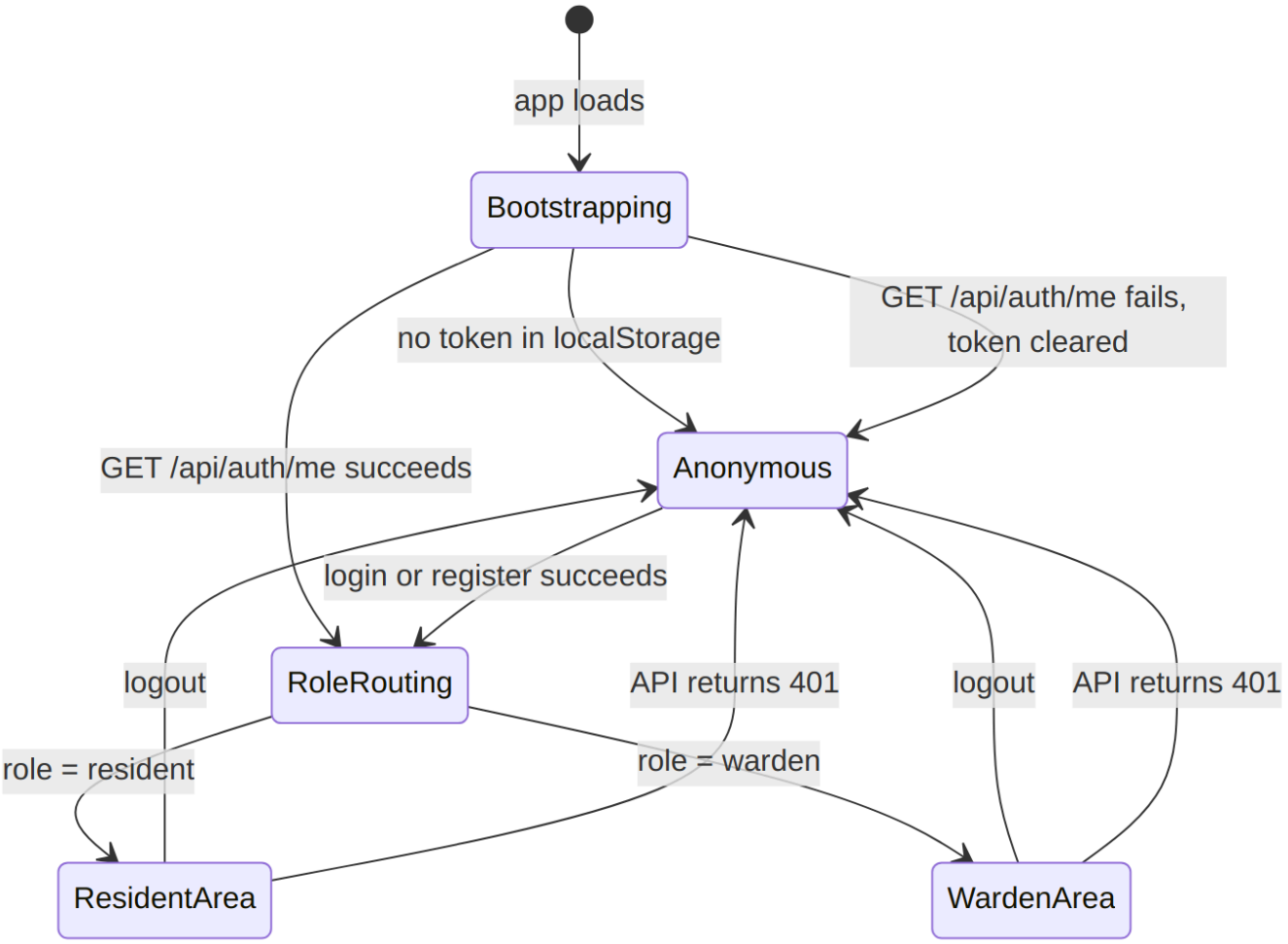




6.3 Client Session (Browser State)

This describes the React application's authentication state, managed by `AuthContext.jsx`.


app loads
Bootstrapping
no token in localStorage
GET /api/auth/me fails, token cleared
Anonymous
GET /api/auth/me succeeds
login or register succeeds
logout
API returns 401
logout
RoleRouting
API returns 401
role = resident
role = warden
ResidentArea
WardenArea

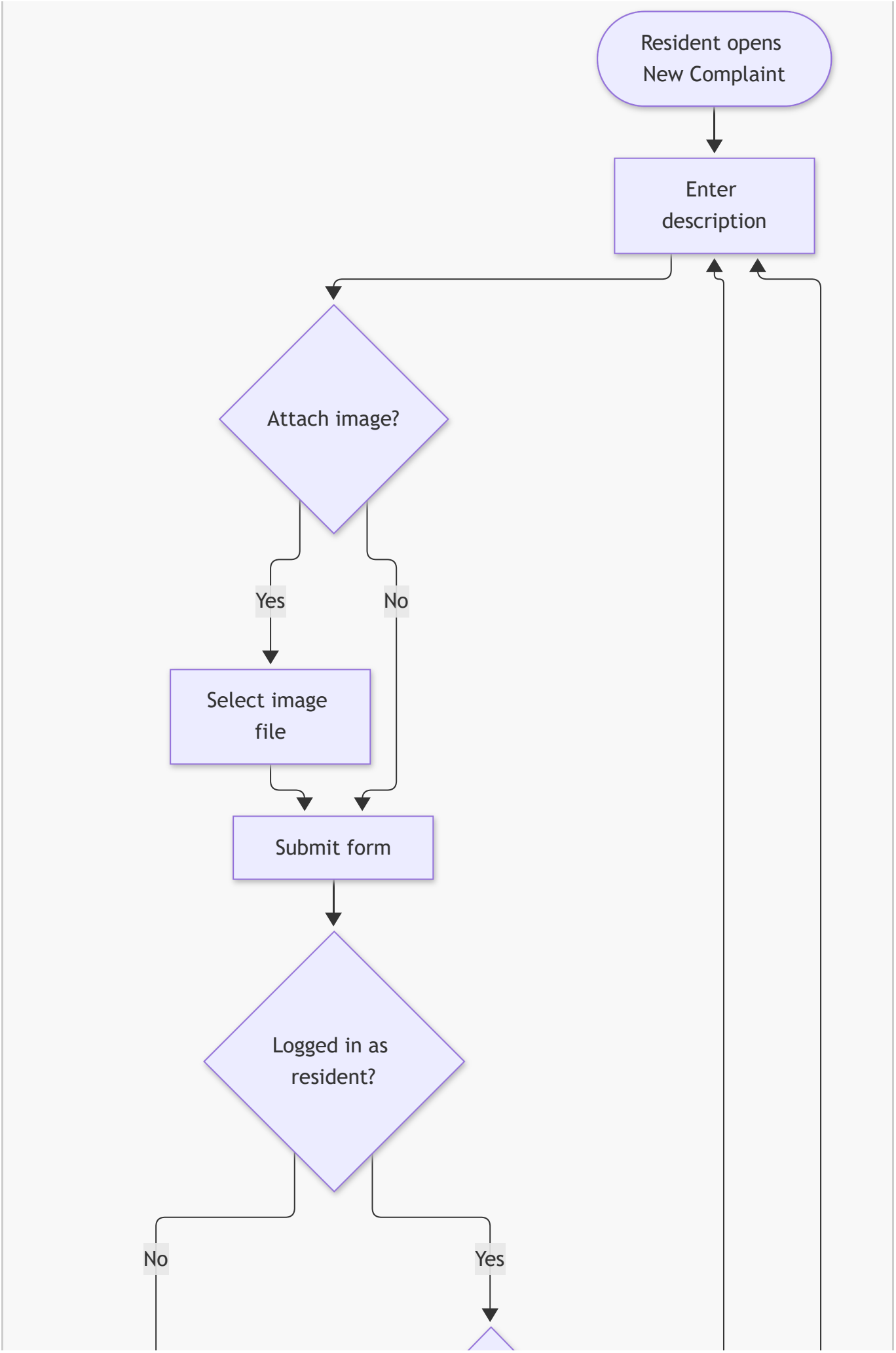


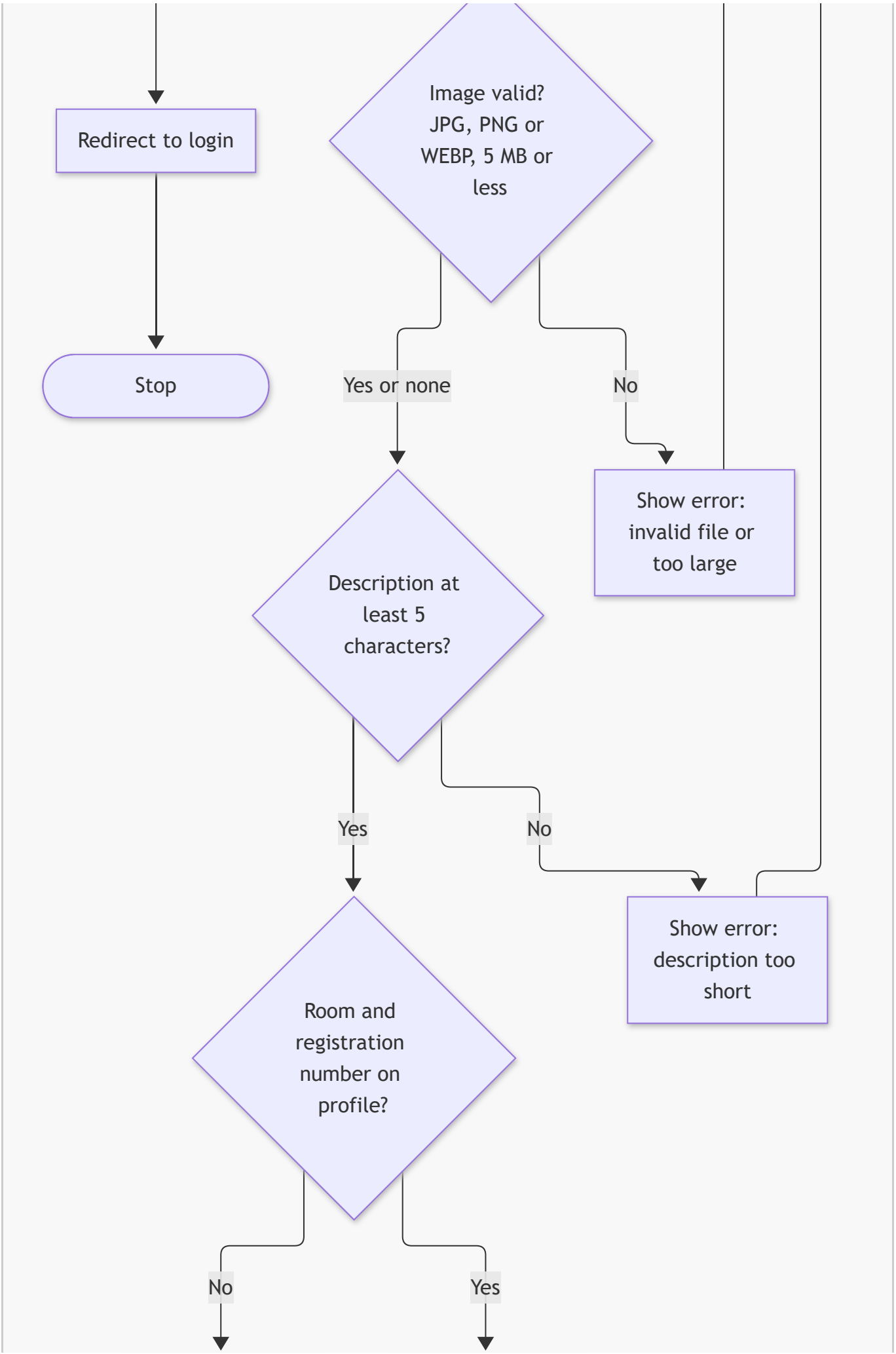
---

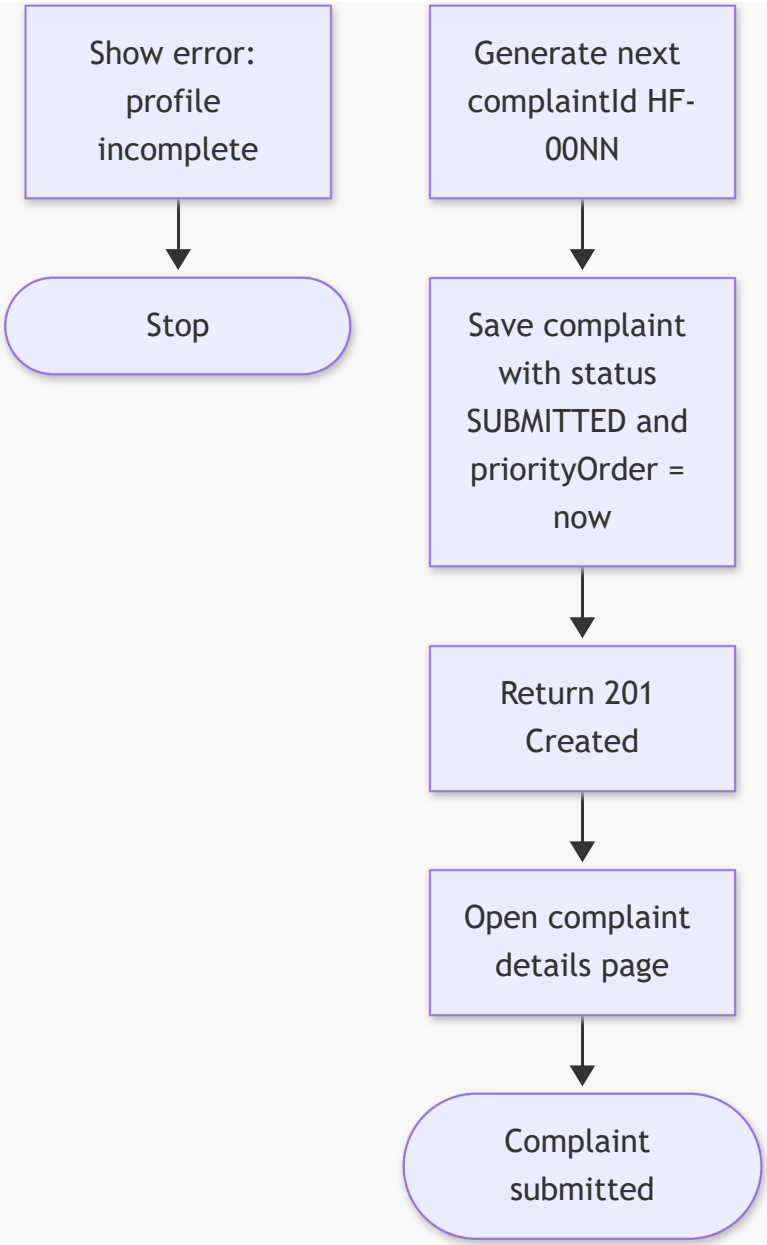
7. Activity Diagrams

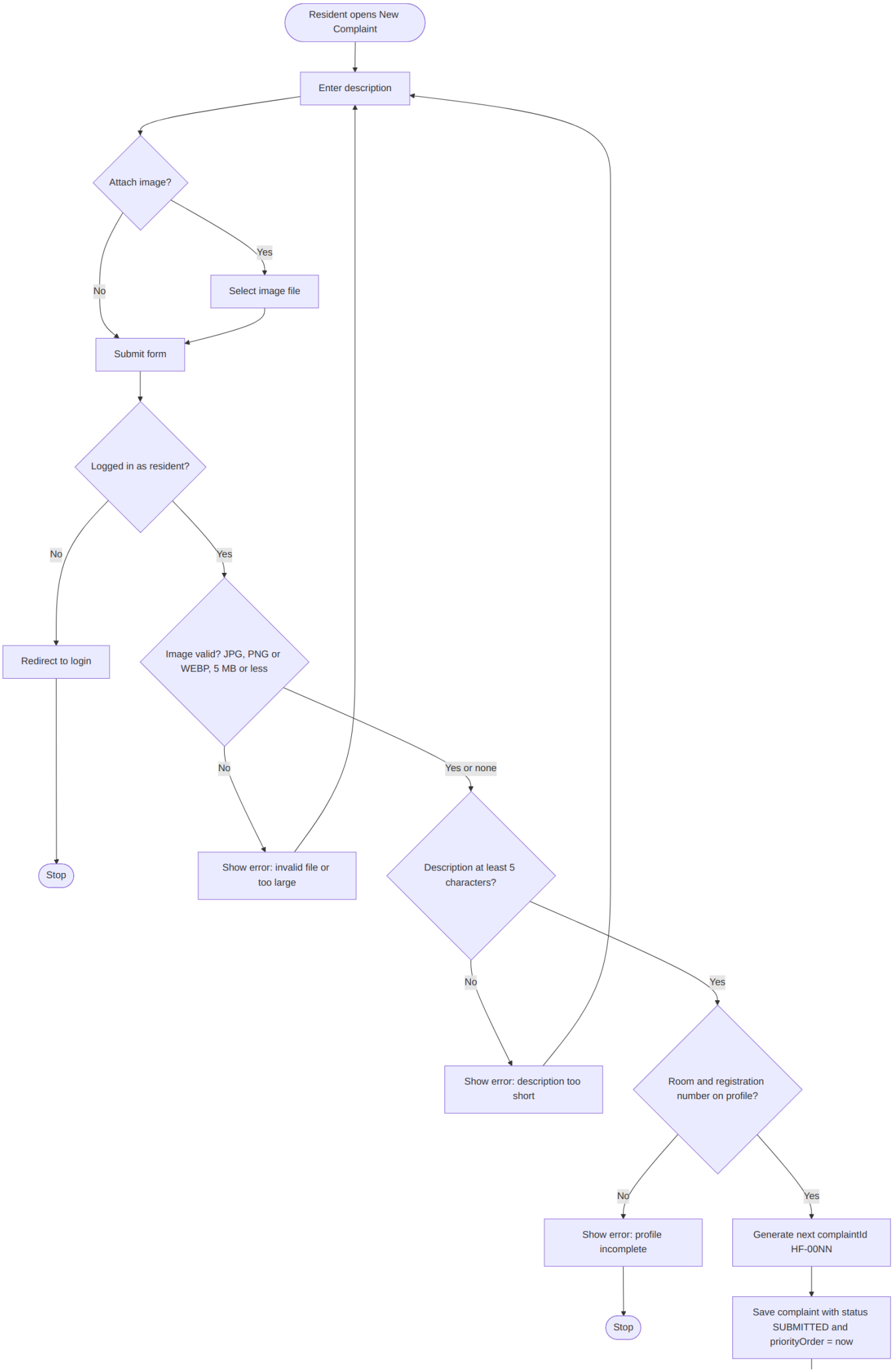
Activity diagrams use `flowchart`. Swimlanes are approximated with subgraphs, one per actor or system component.

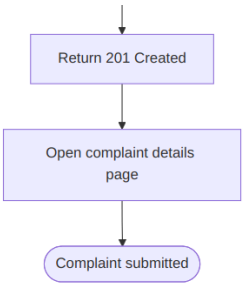
7.1 Activity: Resident Submits Complaint


Resident opens New Complaint
Enter description
Attach image?
Yes
No
Select image file
Submit form
Logged in as resident?
No
Yes

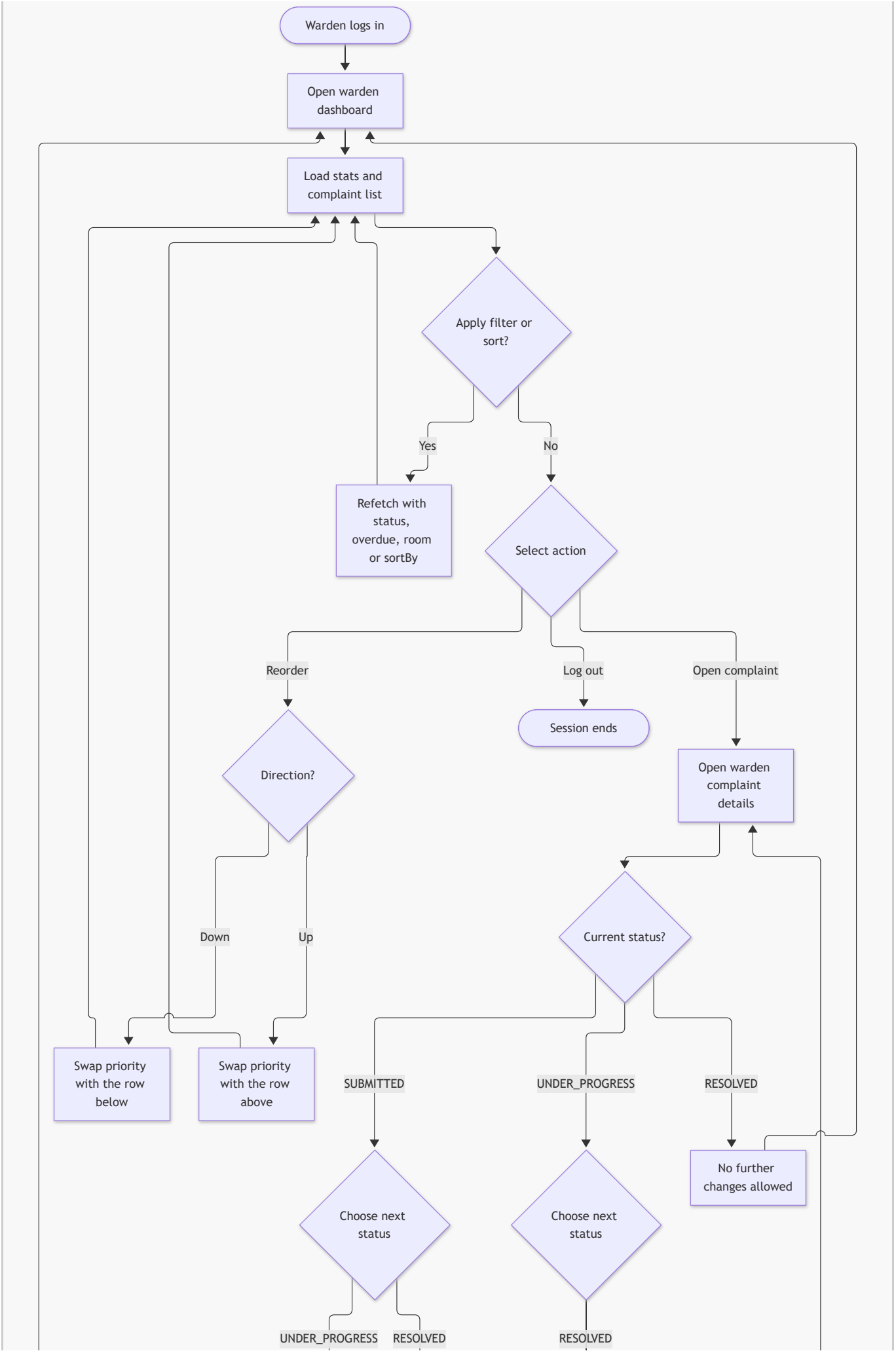

Redirect to login
Stop
Image valid? JPG, PNG or WEBP, 5 MB or less
Yes or none
No
Show error: invalid file or too large
Description at least 5 characters?
Yes
No
Show error: description too short
Room and registration number on profile?
No
Yes

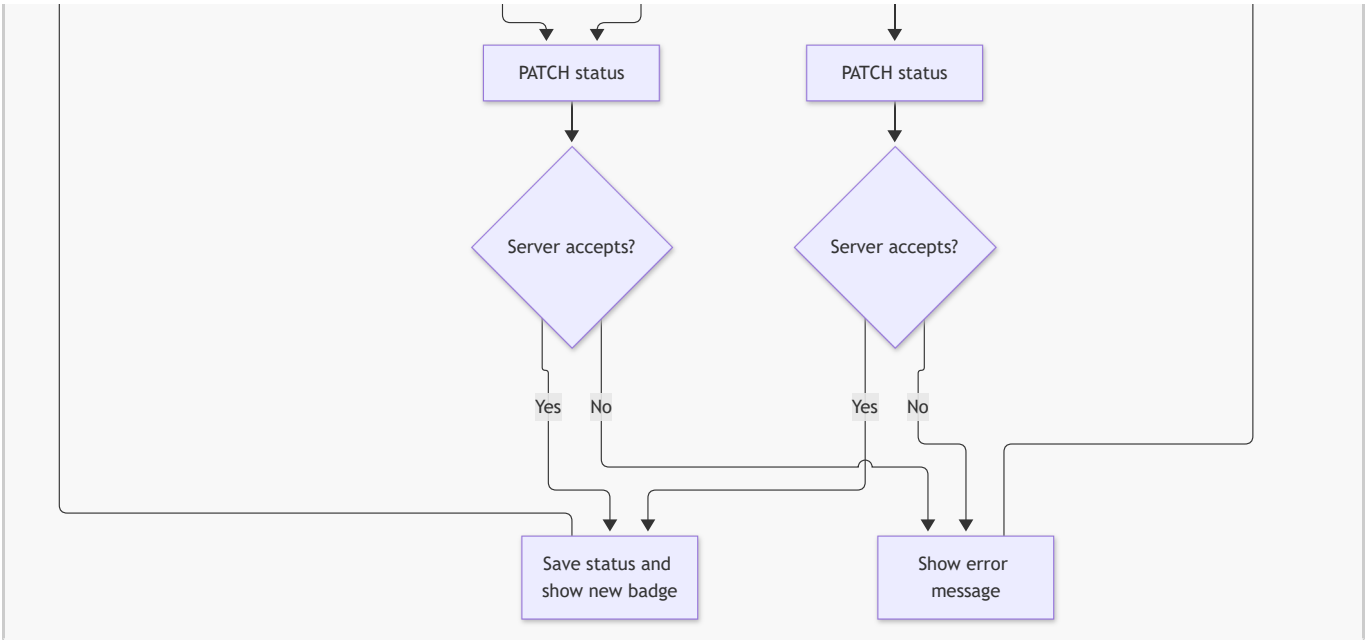

Show error: profile incomplete
Stop
Generate next complaintId HF-00NN
Save complaint with status SUBMITTED and priorityOrder = now
Return 201 Created
Open complaint details page
Complaint submitted

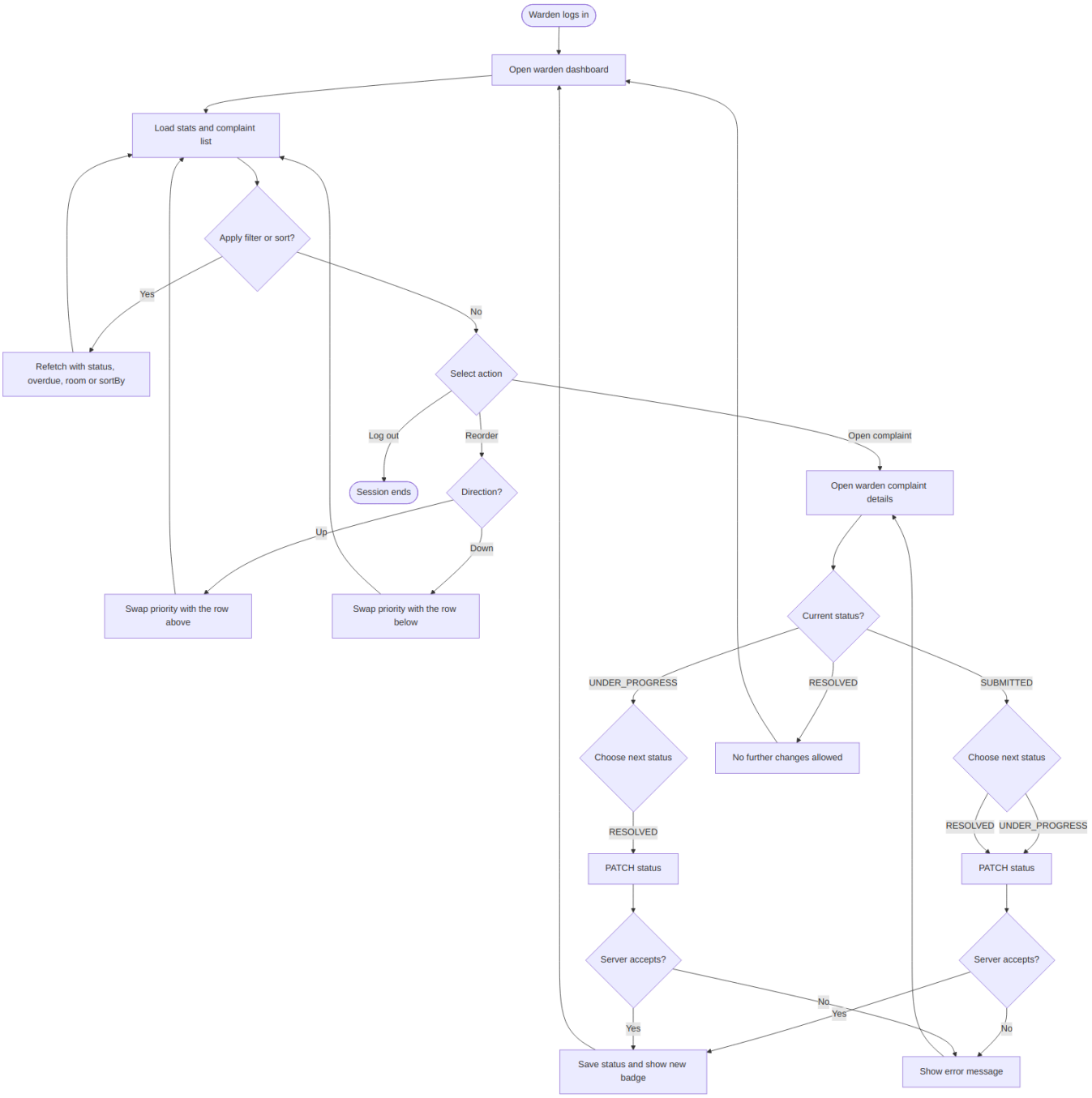

Resident opens New Complaint
Enter description
Attach image?
Yes
No
Select image file
Submit form
Logged in as resident?
No
Yes
Redirect to login
Stop
Image valid? JPG, PNG or WEBP, 5 MB or less
No
Show error: invalid file or too large
Yes or none
Description at least 5 characters?
No
Show error: description too short
Yes
Room and registration number on profile?
No
Show error: profile incomplete
Stop
Yes
Generate next complaintId HF-00NN
Save complaint with status SUBMITTED and priorityOrder = now



7.2 Activity: Warden Complaint Management

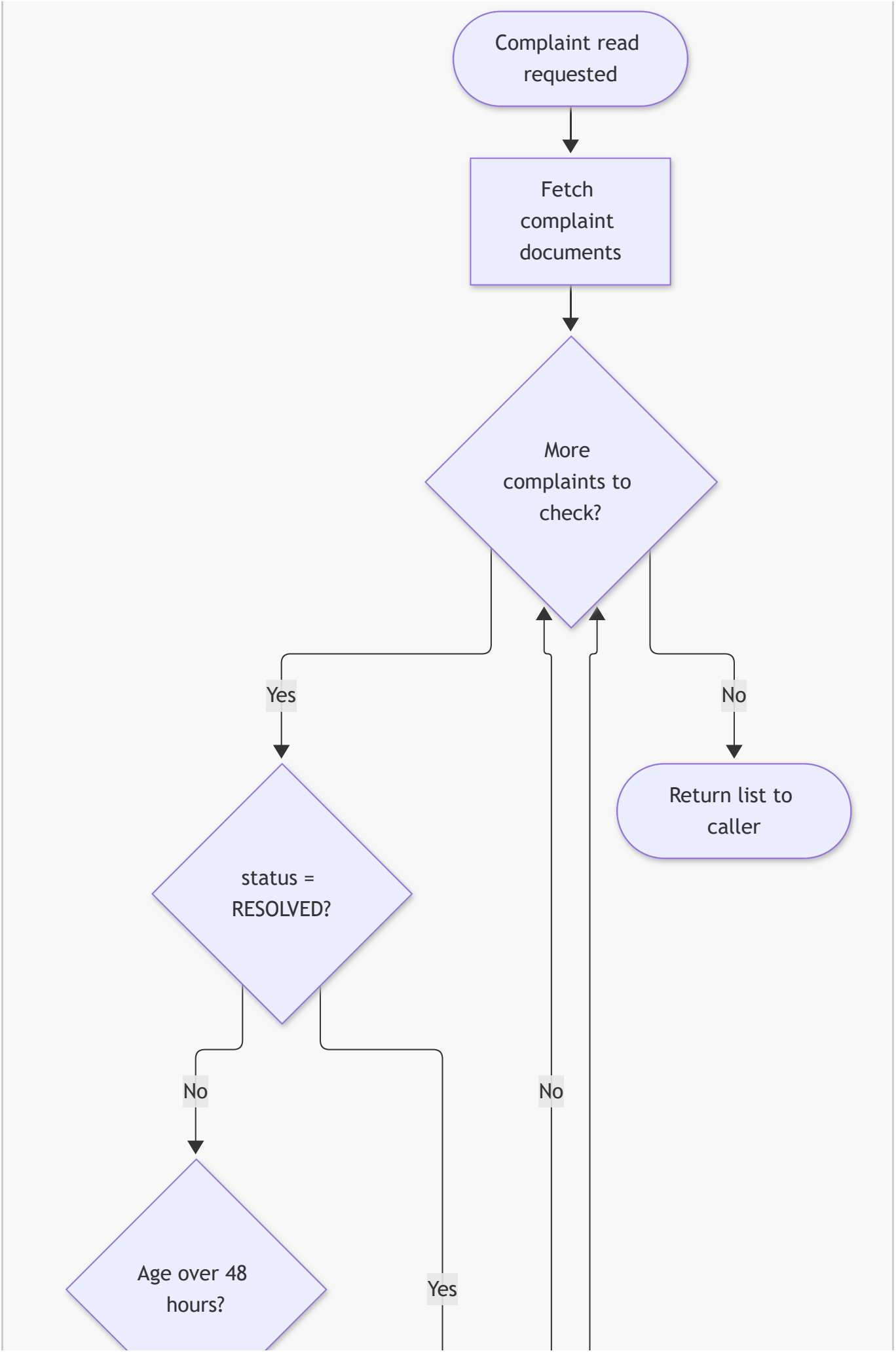

Warden logs in
Open warden dashboard
Load stats and complaint list
Apply filter or sort?
Yes
No
Refetch with status, overdue, room or sortBy
Select action
Reorder
Log out
Open complaint
Session ends
Direction?
Open warden complaint details
Down
Up
Current status?
Swap priority with the row below
Swap priority with the row above
SUBMITTED
UNDER_PROGRESS
RESOLVED
No further changes allowed
Choose next status
Choose next status
UNDER_PROGRESS
RESOLVED
RESOLVED

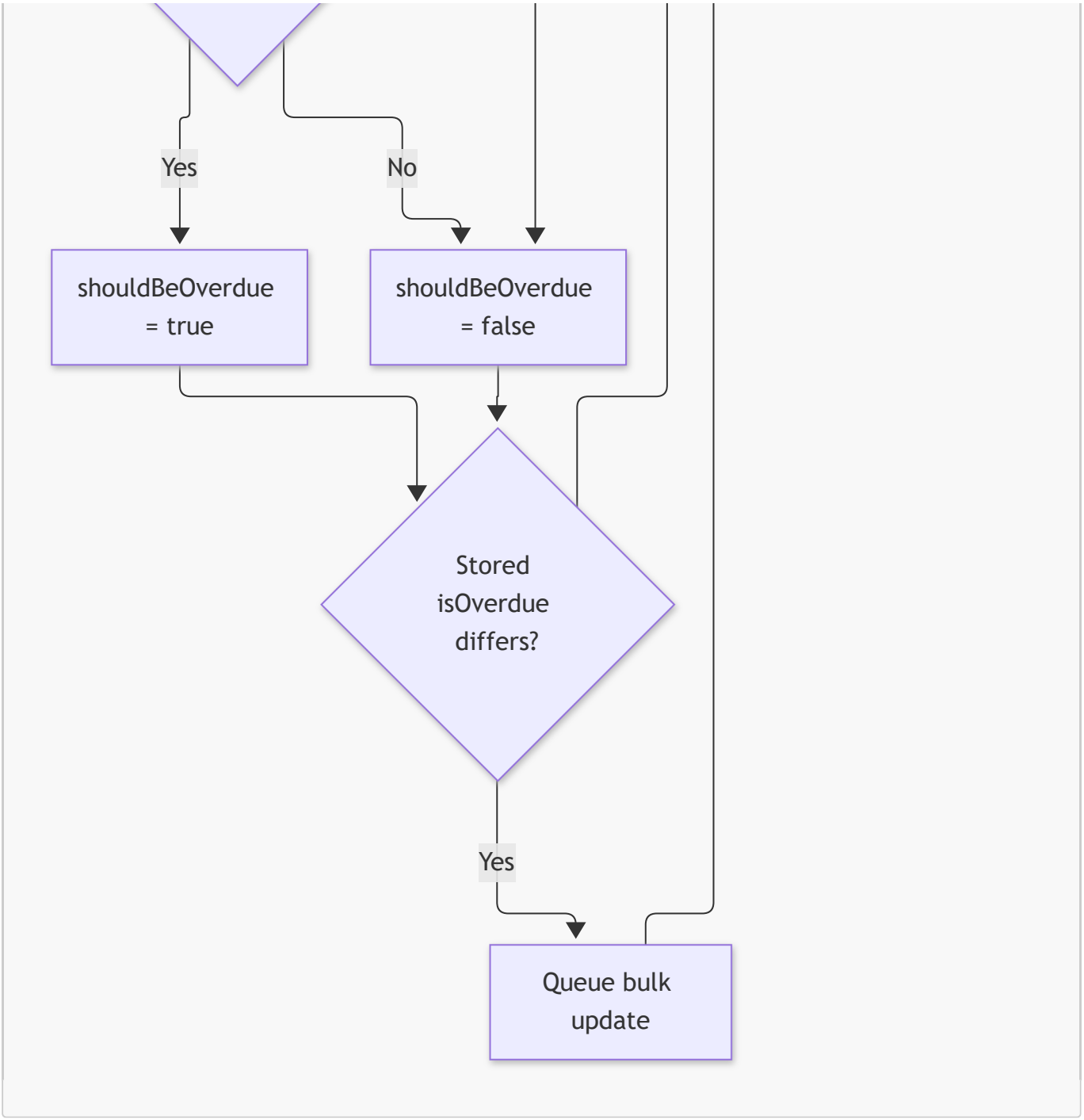

PATCH status
PATCH status
Server accepts?
Server accepts?
Yes
No
Yes
No
Save status and show new badge
Show error message

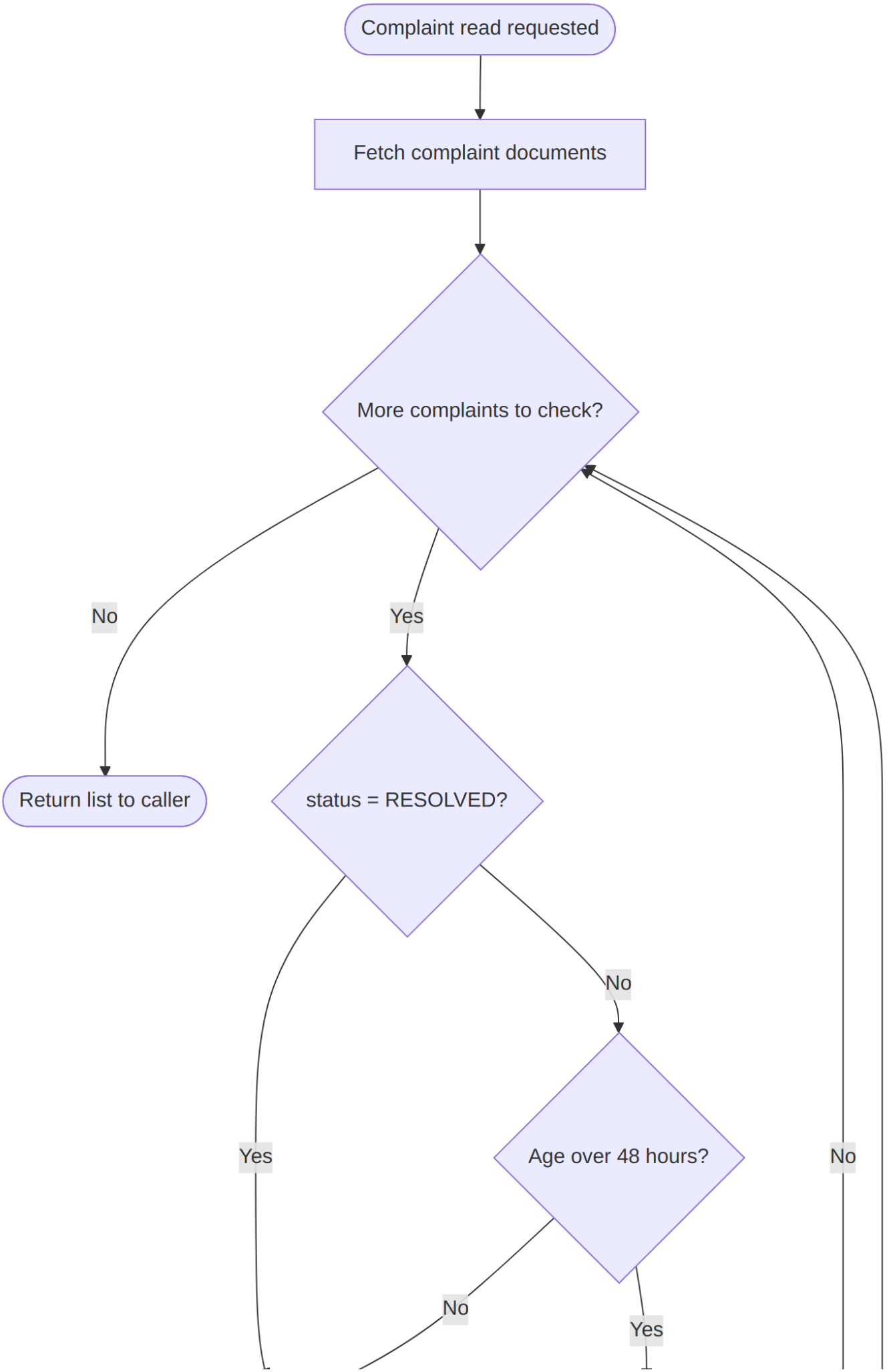


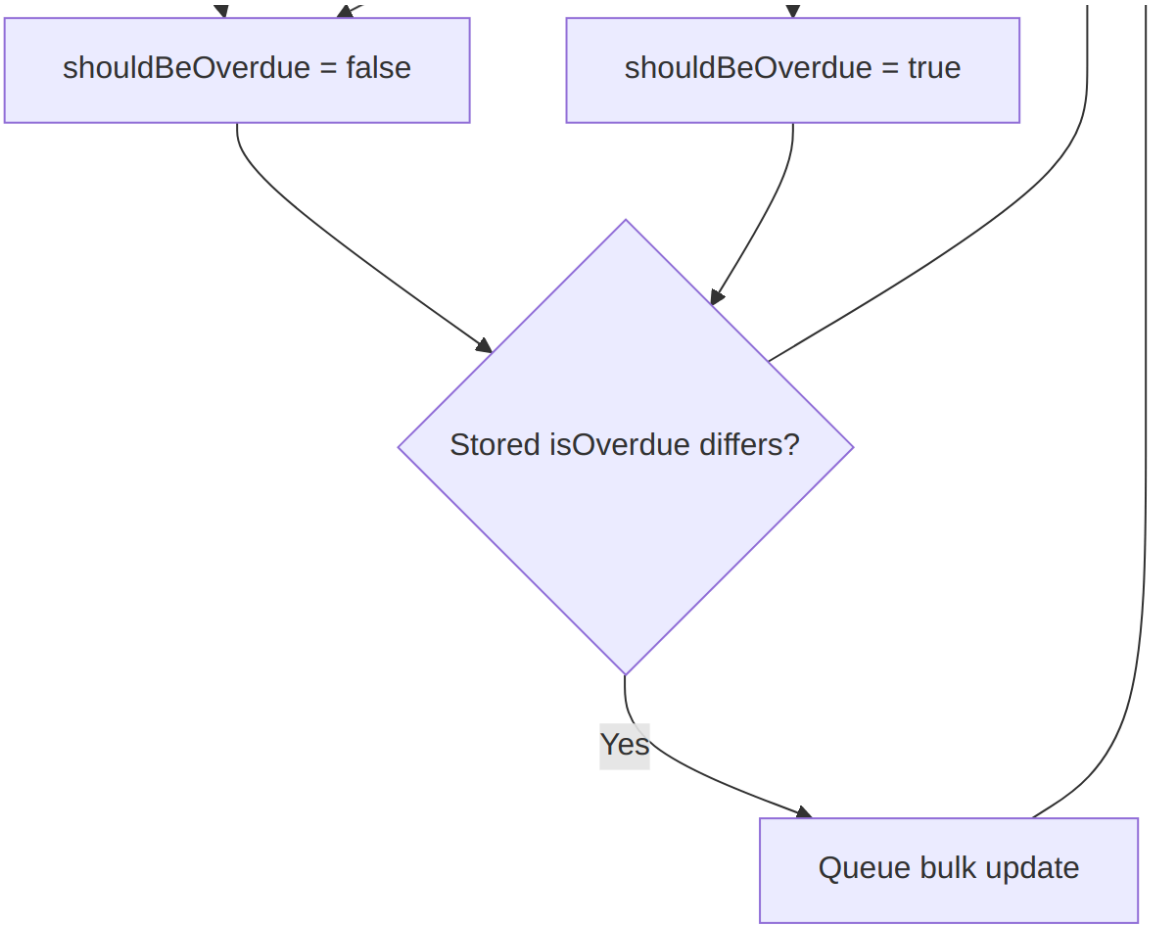
7.3 Activity: Complaint Overdue Evaluation

Runs on every complaint read, whether by resident list, warden list or a single detail request.

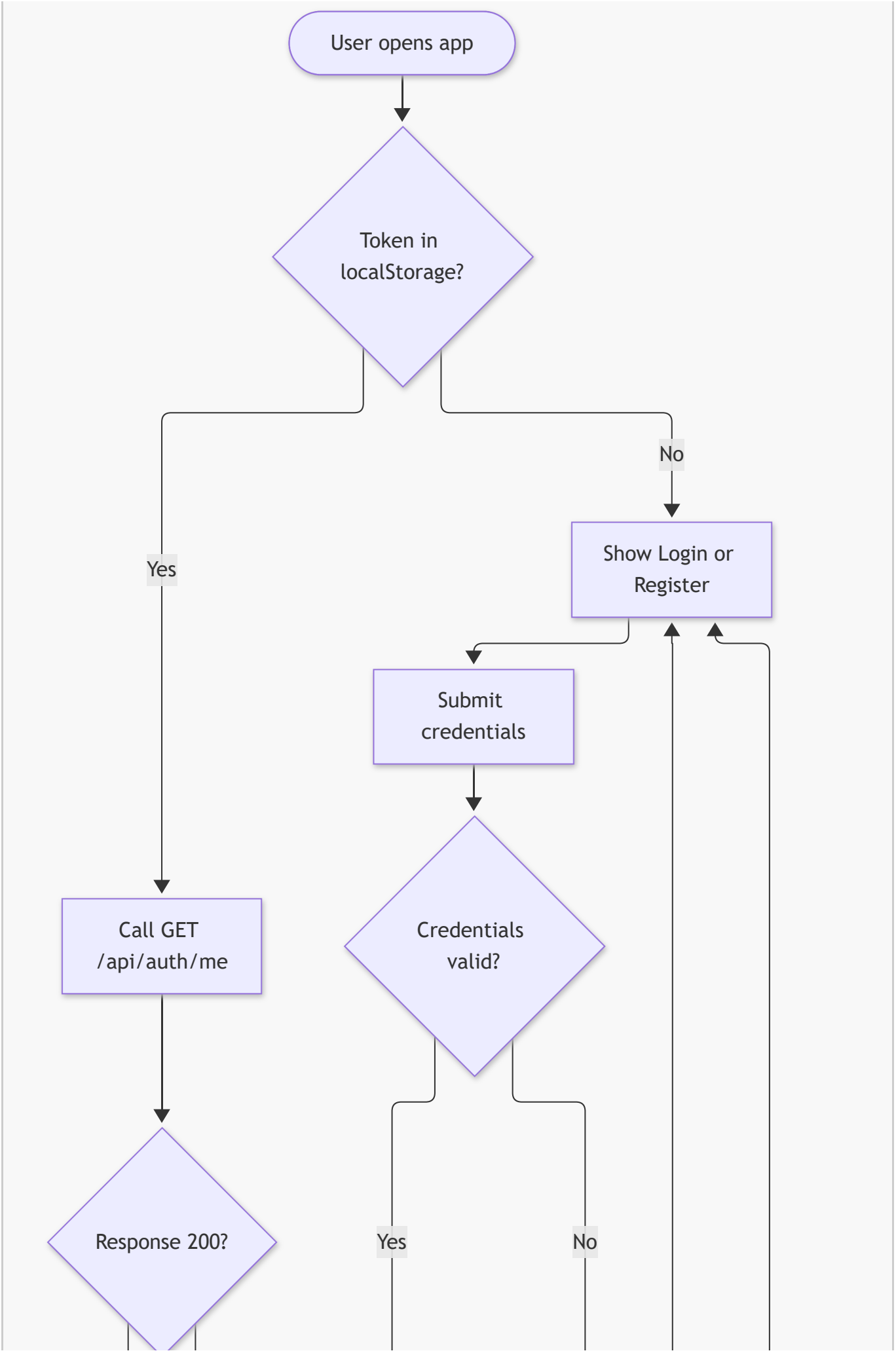

Complaint read requested
Fetch complaint documents
More complaints to check?
Yes
No
Return list to caller
status = RESOLVED?
No
No
Age over 48 hours?
Yes

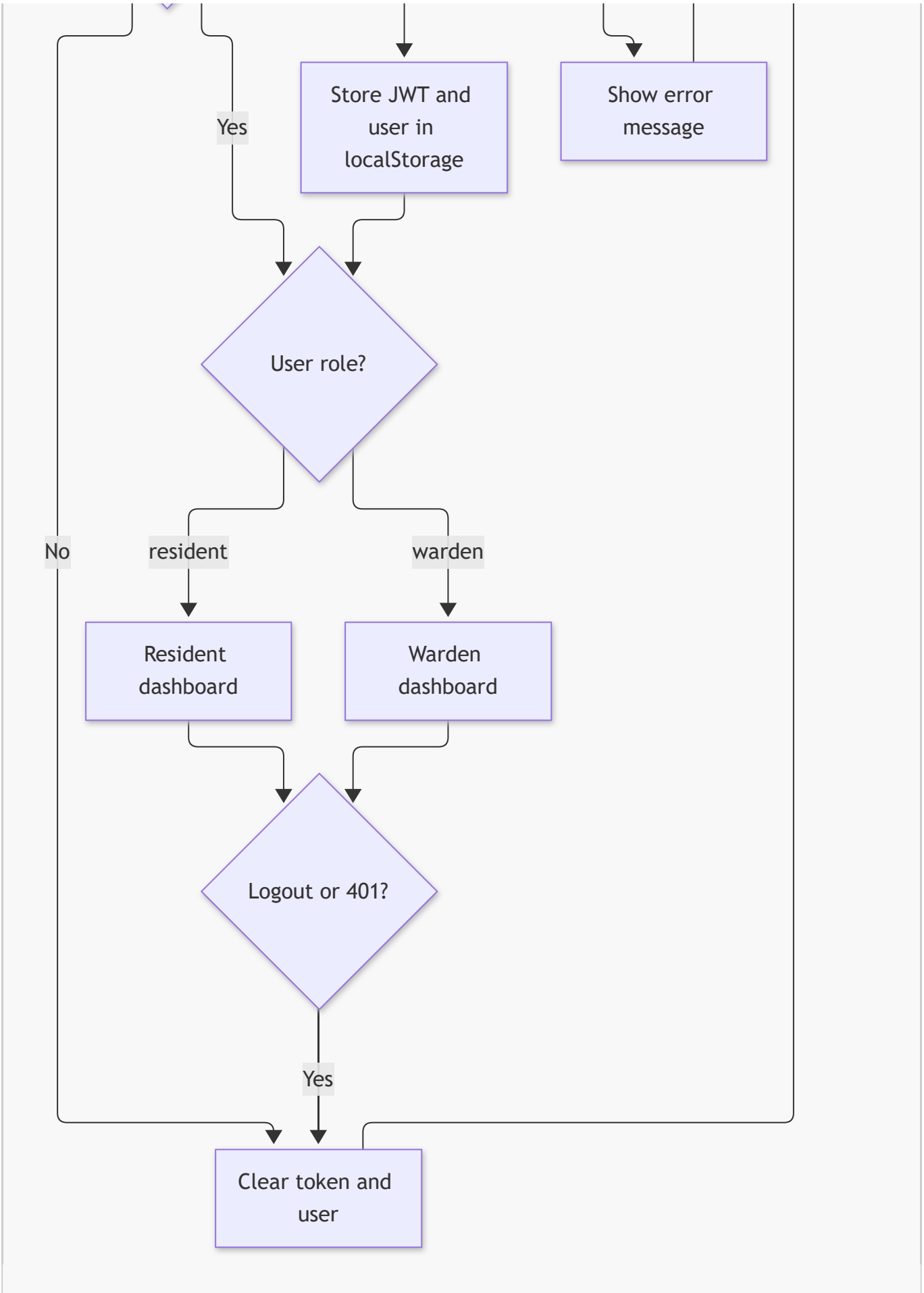

Yes
No
shouldBeOverdue = true
shouldBeOverdue = false
Stored isOverdue differs?
Yes
Queue bulk update

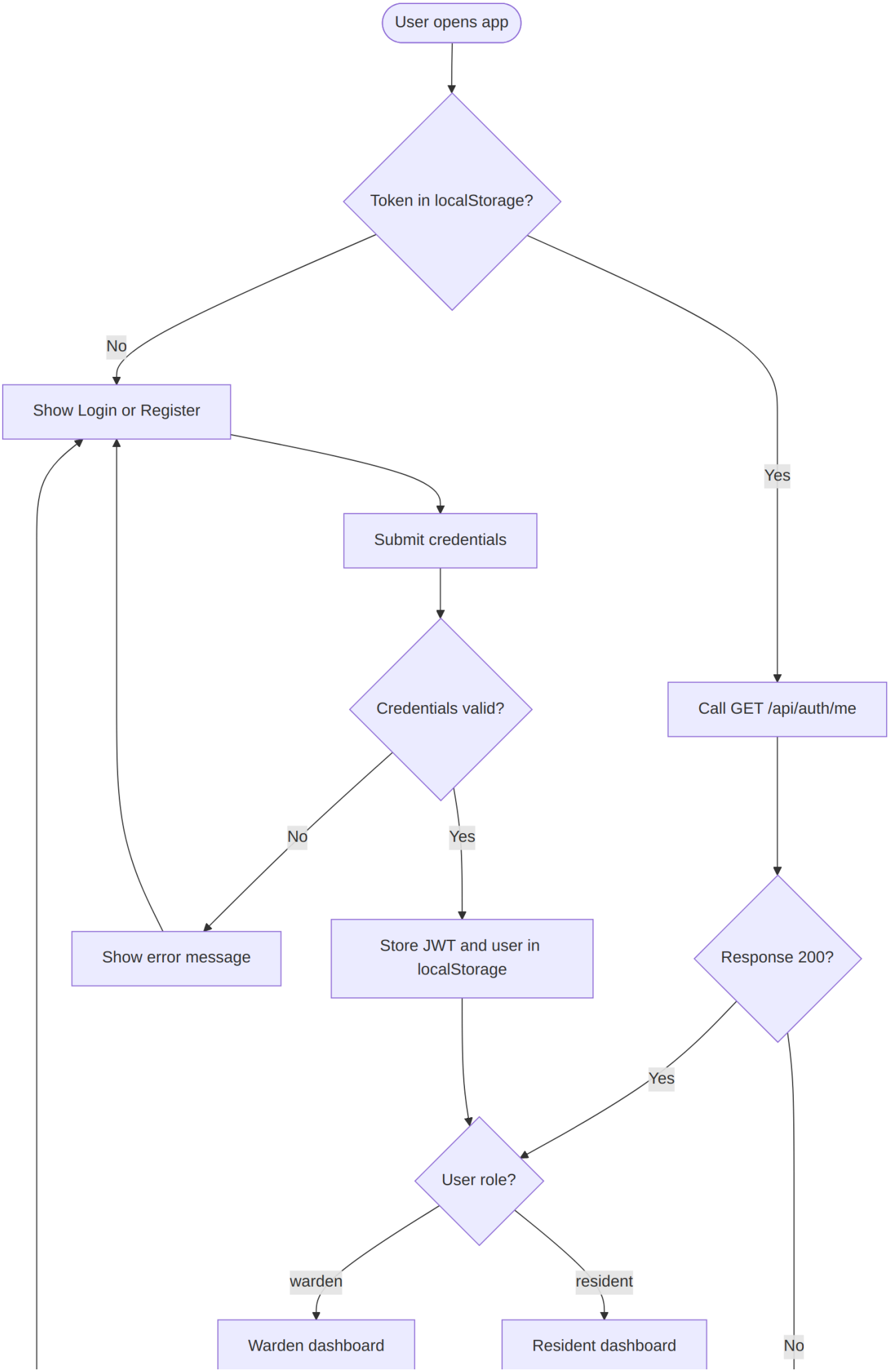

Complaint read requested
Fetch complaint documents
More complaints to check?
No
Yes
Return list to caller
status = RESOLVED?
No
Yes
Age over 48 hours?
No
No
Yes

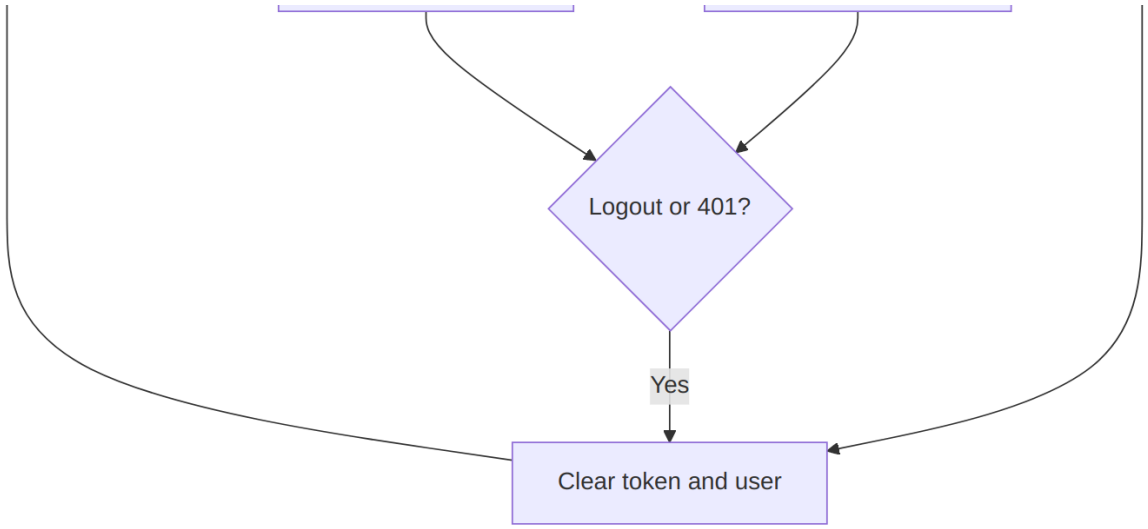


7.4 Activity: Authentication and Role Routing


User opens app
Token in localStorage?
Yes
No
Call GET /api/auth/me
Response 200?
Show Login or Register
Submit credentials
Credentials valid?
Yes
No


Yes
Store JWT and user in localStorage
Show error message
User role?
No
resident
warden
Resident dashboard
Warden dashboard
Logout or 401?
Yes
Clear token and user


User opens app
Token in localStorage?
No
Yes
Show Login or Register
Submit credentials
Credentials valid?
No
Yes
Show error message
Store JWT and user in localStorage
Call GET /api/auth/me
Response 200?
Yes
No
User role?
warden
resident
Warden dashboard
Resident dashboard



---

8. Diagram-to-Source Mapping

Diagram	Primary source files
Use Case	`server/routes/*.js`, `client/src/App.jsx`, `Report/02_Requirements/Use_Cases_and_User_Stories.md`
Class (domain)	`server/models/User.js`, `Complaint.js`, `Feedback.js`, `Counter.js`
Class (layered)	`server/routes`, `controllers`, `services`, `middleware`, `utils`
Object	Schema definitions in `server/models`, seed data in `server/seed/seed.js`
Sequence: Login	`client/src/context/AuthContext.jsx`, `server/controllers/authController.js`, `server/services/authService.js`
Sequence: Submit	`client/src/pages/NewComplaint.jsx`, `server/middleware/upload.js`, `server/services/complaintService.js`
Sequence: Status update	`client/src/pages/WardenComplaintDetails.jsx`, `server/services/complaintService.js` (`ALLOWED_TRANSITIONS`)
Sequence: Reorder	`client/src/pages/WardenDashboard.jsx`, `server/services/complaintService.js` (`reorderComplaint`)
Sequence: View own	`client/src/pages/ResidentDashboard.jsx`, `server/utils/overdue.js`
Sequence: Feedback	`server/services/feedbackService.js`, `client/src/pages/ComplaintDetails.jsx`
Collaboration	Same files as the matching sequence diagrams
State: Complaint	`server/services/complaintService.js`, `server/models/Complaint.js`
State: Overdue	`server/utils/overdue.js`
State: Client session	`client/src/context/AuthContext.jsx`, `client/src/api/axios.js`

Diagram	Primary source files
Activity: Submit	`client/src/pages/NewComplaint.jsx`, `server/services/complaintService.js`
Activity: Warden	`client/src/pages/WardenDashboard.jsx`, `WardenComplaintDetails.jsx`
Activity: Overdue	`server/utils/overdue.js`
Activity: Auth	`client/src/context/AuthContext.jsx`, `client/src/components/ProtectedRoute.jsx`